COMP 203: การคำนวณแบบขนานและแบบกระจาย
อัลกอริทึมบน PRAM

COMP 203: Parallel and Distributed Computing — PRAM Algorithms

Siddhartha Chatterjee
Jan Prins

ภาคฤดูใบไม้ผลิ 2002

สารบัญ

1 แบบจำลองการคำนวณ PRAM **2**

2 กระบวนทัศน์งาน–เวลา (Work-Time) **5**
2.1 ทฤษฎีบทของเบรนต์ . . . 6
2.2 การออกแบบอัลกอริทึมแบบขนานที่ดี . . . 7

3 เทคนิคพื้นฐานในการออกแบบอัลกอริทึมบน PRAM **9**
3.1 ต้นไม้สมดุล . . . 9
3.2 การกระโดดตัวชี้ . . . 11
3.3 การต่อเรียงอัลกอริทึม . . . 13
3.4 ทัวร์ออยเลอร์ . . . 15
3.5 การแบ่งแยกและเอาชนะ . . . 17
3.6 การทำลายความสมมาตร . . . 18

4 ทัวร์อัลกอริทึมแบบขนานเชิงข้อมูล **21**
4.1 อัลกอริทึมพื้นฐานที่ใช้ scan . . . 21
4.2 การวิเคราะห์ศัพท์แบบขนาน . . . 22

5 พลังเชิงเปรียบเทียบของแบบจำลอง PRAM **24**
5.1 พลังของการอ่านพร้อมกัน . . . 24
5.2 พลังของการเขียนพร้อมกัน . . . 24
5.3 การวัดเชิงปริมาณของพลังการเข้าถึงหน่วยความจำพร้อมกัน . . . 25
5.4 ความสัมพันธ์ระหว่างแบบจำลอง PRAM กับการคำนวณแบบขนานในทางปฏิบัติ . . . 27

ภาคผนวก: รายการแก้คำผิดจากต้นฉบับ **28**

1 แบบจำลองการคำนวณ PRAM

ในหน่วยแรกของวิชานี้ เราจะศึกษาอัลกอริทึมแบบขนาน (parallel algorithms) ในบริบทของแบบจำลองการคำนวณแบบขนานที่เรียกว่า Parallel Random Access Machine (PRAM) ดังที่ชื่อบอกไว้ แบบจำลอง PRAM เป็นส่วนขยายของแบบจำลอง RAM ซึ่งเป็นแบบจำลองการคำนวณแบบลำดับ (sequential) ที่คุ้นเคยกันดีและใช้ในการวิเคราะห์อัลกอริทึม เราจะใช้ PRAM แบบประสานจังหวะ (synchronous PRAM) ซึ่งนิยามไว้ดังนี้

1. มีโพรเซสเซอร์ p ตัวเชื่อมต่อกับหน่วยความจำร่วม (shared memory) เพียงชุดเดียว
2. โพรเซสเซอร์แต่ละตัวมีดัชนีเฉพาะตัว $1 \leqslant i \leqslant p$ เรียกว่ารหัสโพรเซสเซอร์ (processor id)
3. โปรแกรมเดียวถูกประมวลผลในลักษณะคำสั่งสายเดียว ข้อมูลหลายสาย (single-instruction stream, multiple-data stream: SIMD) คำสั่งแต่ละคำสั่งในสายคำสั่งจะถูกดำเนินการโดยโพรเซสเซอร์ทุกตัวพร้อมกัน และใช้เวลาหนึ่งหน่วยเวลา ไม่ว่าจะมีโพรเซสเซอร์กี่ตัวก็ตาม
4. โพรเซสเซอร์แต่ละตัวมีแฟล็กส่วนตัว (private flag) ที่ควบคุมว่าโพรเซสเซอร์นั้นจะทำงาน (active) ในการประมวลผลคำสั่งหรือไม่ โพรเซสเซอร์ที่ไม่ทำงานจะไม่เข้าร่วมประมวลผลคำสั่งใด ๆ ยกเว้นคำสั่งที่ใช้รีเซ็ตแฟล็ก

รหัสโพรเซสเซอร์ใช้แยกพฤติกรรมของโพรเซสเซอร์แต่ละตัวขณะประมวลผลโปรแกรมร่วมกันได้ ตัวอย่างเช่น โพรเซสเซอร์แต่ละตัวอาจใช้รหัสของตนสร้างที่อยู่ (address) ที่ไม่ซ้ำกันในหน่วยความจำร่วมเพื่ออ่านค่าจากตำแหน่งนั้น ลำดับคำสั่งชุดหนึ่งอาจถูกประมวลผลแบบมีเงื่อนไขโดยโพรเซสเซอร์เพียงกลุ่มย่อย โดยโพรเซสเซอร์ทุกตัวจะประเมินเงื่อนไขแล้วใช้ผลนั้นตั้งค่าแฟล็กส่วนตัว เฉพาะโพรเซสเซอร์ที่ทำงานอยู่เท่านั้นที่จะดำเนินการคำสั่งที่ตามมา เมื่อจบลำดับคำสั่งนั้น แฟล็กส่วนตัวจะถูกรีเซ็ตเพื่อให้โพรเซสเซอร์ทุกตัวกลับมาทำงานต่อ

การทำงานของ PRAM แบบประสานจังหวะอาจทำให้โพรเซสเซอร์หลายตัวเข้าถึงตำแหน่งเดียวกันในหน่วยความจำร่วมพร้อมกัน แบบจำลอง PRAM ของเรามีหลายรูปแบบย่อย ขึ้นอยู่กับว่าอนุญาตให้เข้าถึงพร้อมกันเช่นนี้ (การเข้าถึงพร้อมกัน: concurrent access) หรือห้าม (การเข้าถึงแบบเอกสิทธิ์: exclusive access) และเนื่องจากการเข้าถึงอาจเป็นการอ่านหรือการเขียน จึงมีความเป็นไปได้สี่แบบดังนี้

1. *อ่านแบบเอกสิทธิ์ เขียนแบบเอกสิทธิ์* (Exclusive Read Exclusive Write: EREW): PRAM รูปแบบนี้ไม่อนุญาตให้เข้าถึงตำแหน่งหน่วยความจำเดียวกันพร้อมกันในลักษณะใดเลย โปรแกรมที่ถูกต้องทุกโปรแกรมสำหรับ PRAM แบบนี้ต้องรับประกันว่าไม่มีโพรเซสเซอร์สองตัวใดเข้าถึงตำแหน่งหน่วยความจำเดียวกันในหน่วยเวลาเดียวกัน
2. *อ่านพร้อมกัน เขียนแบบเอกสิทธิ์* (Concurrent Read Exclusive Write: CREW): PRAM รูปแบบนี้อนุญาตให้อ่านพร้อมกันได้ แต่ไม่อนุญาตให้เขียนพร้อมกันลงในตำแหน่งหน่วยความจำร่วม โพรเซสเซอร์ทุกตัวที่อ่านตำแหน่งหน่วยความจำเดียวกันพร้อมกันจะได้ค่าเดียวกัน
3. *อ่านแบบเอกสิทธิ์ เขียนพร้อมกัน* (Exclusive Read Concurrent Write: ERCW): PRAM รูปแบบนี้อนุญาตให้เขียนพร้อมกันได้ แต่ไม่อนุญาตให้อ่านพร้อมกันจากตำแหน่งหน่วยความจำร่วม โดยทั่วไปไม่นิยมพิจารณารูปแบบนี้แยกต่างหาก เพราะถือว่าถูกรวมอยู่ในรูปแบบถัดไปแล้ว
4. *อ่านพร้อมกัน เขียนพร้อมกัน* (Concurrent Read Concurrent Write: CRCW): PRAM รูปแบบนี้อนุญาตให้ทั้งอ่านและเขียนพร้อมกันในตำแหน่งหน่วยความจำร่วม รูปแบบนี้ยังแบ่งเป็นรูปแบบย่อยได้อีกหลายแบบ ขึ้นอยู่กับวิธีตัดสินผลของการเขียนพร้อมกัน
   (a) *Common CRCW*: แบบจำลองนี้อนุญาตให้เขียนพร้อมกันได้ก็ต่อเมื่อโพรเซสเซอร์ทุกตัวพยายามเขียนค่าเดียวกัน (ซึ่งจะกลายเป็นค่าที่ถูกเก็บ)
   (b) *Arbitrary CRCW*: ในแบบจำลองนี้ ค่าที่ถูกเก็บคือค่าหนึ่งที่เลือกมาโดยพลการจากค่าทั้งหมดที่ถูกเขียนลงตำแหน่งหน่วยความจำนั้น
   (c) *Priority CRCW*: ในแบบจำลองนี้ ค่าที่ถูกเก็บคือค่าที่เขียนโดยโพรเซสเซอร์ที่มีรหัสน้อยที่สุดในบรรดาโพรเซสเซอร์ที่เขียนลงตำแหน่งนั้น

(d) *Combining CRCW*: ในแบบจำลองนี้ ค่าที่ถูกเก็บคือผลรวม (combination) ของค่าทั้งหมดที่ถูกเขียน (โดยทั่วไปรวมด้วยตัวดำเนินการที่มีสมบัติการเปลี่ยนหมู่และการสลับที่ เช่น $+$ หรือ $\max$)

แบบจำลองต่าง ๆ เหล่านี้แทนข้อจำกัดที่แตกต่างกันในการออกแบบอัลกอริทึม ความแตกต่างไม่ได้อยู่ที่พลังในการแสดงออก (expressive power) แต่อยู่ในแง่ของทฤษฎีความซับซ้อน (complexity-theoretic) เราจะพิจารณาประเด็นนี้เพิ่มเติมในหัวข้อ 5

เราศึกษาอัลกอริทึมบน PRAM ด้วยเหตุผลหลายประการ

1. มีงานวิจัยจำนวนมากที่พัฒนามาอย่างดีแล้วเกี่ยวกับการออกแบบอัลกอริทึมบน PRAM และความซับซ้อนของอัลกอริทึมเหล่านั้น
2. แบบจำลอง PRAM มุ่งเน้นเฉพาะประเด็นภาวะพร้อมกัน (concurrency) และจงใจละเลยประเด็นการประสานจังหวะ (synchronization) และการสื่อสาร (communication) จึงทำหน้าที่เป็นแบบจำลองพื้นฐาน (baseline) ของภาวะพร้อมกัน กล่าวอีกนัยหนึ่ง ถ้าคุณหาอัลกอริทึมแบบขนานที่ดีบนแบบจำลอง PRAM ไม่ได้ คุณก็จะไม่ได้อัลกอริทึมแบบขนานที่ดีในโลกจริงเช่นกัน
3. แบบจำลองนี้ชัดแจ้ง (explicit): เราต้องระบุการดำเนินการที่ทำในแต่ละขั้น และการจัดตารางการดำเนินการลงบนโพรเซสเซอร์
4. เป็นกระบวนทัศน์การออกแบบที่แข็งแรง (robust) อัลกอริทึมจำนวนมากสำหรับแบบจำลองอื่น (เช่น แบบจำลองเครือข่าย: network model) สามารถพัฒนาต่อได้โดยตรงจากอัลกอริทึมบน PRAM

Claude อธิบายเพิ่ม: วิธีอ่านเอกสารนี้

กล่องสีส้มแบบนี้เป็น**ส่วนที่ Claude เขียนเพิ่มเติม** เพื่อสรุป ยกตัวอย่าง และเชื่อมโยงเนื้อหา ไม่ใช่ส่วนของต้นฉบับ เนื้อหานอกกล่องทั้งหมดแปลจากต้นฉบับครบถ้วน รูปที่ 1–6 นำมาจากต้นฉบับโดยตรง (แปลเฉพาะคำบรรยายใต้รูป) ส่วนอัลกอริทึมและตารางตัวอย่างพิมพ์ใหม่ตามต้นฉบับ โดยคงคำสงวนของ pseudocode (forall, do, enddo ฯลฯ) ไว้เป็นภาษาอังกฤษ
คำผิดในต้นฉบับ: จุดที่ต้นฉบับพิมพ์ผิดได้แก้ไว้แล้ว และมีเครื่องหมาย [แก้ n] กำกับ คลิกเพื่อดูว่าต้นฉบับเขียนว่าอะไร ฉบับที่ถูกคืออะไร และเพราะอะไร (รวมไว้ในภาคผนวกท้ายเล่ม)
จำง่าย ๆ: ชื่อรุ่นของ PRAM อ่านเป็นคู่ “การอ่าน/การเขียน” เช่น CREW = อ่านพร้อมกันได้ (CR) แต่เขียนพร้อมกันไม่ได้ (EW) ความสามารถเรียงจากน้อยไปมากคือ EREW $\leq$ CREW $\leq$ CRCW (หัวข้อ 5 จะวัดว่าต่างกันเท่าไร)

ข้อสังเกตแทรก 1. ต่อจากนี้ เราจะใช้คำว่า *เวกเตอร์* (vector) และ *เมทริกซ์* (matrix) ในความหมายตามปกติของพีชคณิตเชิงเส้น และใช้คำว่า *ลำดับ* (sequence) สำหรับรายการเชิงเส้น เราสงวนคำว่า *อาร์เรย์* (array) ไว้สำหรับโครงสร้างข้อมูลเชิงรูปธรรมที่คุ้นเคยกันดี ซึ่งใช้สร้างสิ่งนามธรรมอื่น ๆ ทั้งหมดนี้ โดยทั่วไปอาร์เรย์อาจมีหลายมิติ สัญกรณ์สามส่วน (triplet notation) $\ell : h : s$ เมื่อ $\ell \leqslant h$ และ $s > 0$ แทนเซต $\{\ell + is \mid 0 \leqslant i \leqslant \lfloor \frac{h-\ell}{s} \rfloor\}$ ถ้า $s = 1$ เราจะละไว้ในสัญกรณ์ ดังนั้น $\ell : h \equiv \ell : h : 1$ □

ตัวอย่าง 1 (ผลบวกเวกเตอร์). ในฐานะตัวอย่างแรกของอัลกอริทึมบน PRAM ให้เราคำนวณ $z = v + w$ เมื่อ v, w และ z เป็นเวกเตอร์ยาว n ที่เก็บเป็นอาร์เรย์หนึ่งมิติในหน่วยความจำร่วม เราอธิบายอัลกอริทึมบน PRAM โดยให้โปรแกรมเดียวที่โพรเซสเซอร์ทุกตัวประมวลผล โดยทั่วไปรหัสโพรเซสเซอร์จะปรากฏเป็นตัวแปรของโปรแกรมชื่อ i ซึ่งมีค่าต่างกัน $1 \leqslant i \leqslant p$ ในแต่ละโพรเซสเซอร์ ดังนั้นถ้า $n = p$ โปรแกรมบวกเวกเตอร์ก็มีเพียงคำสั่งเดียวคือ $z[i] \leftarrow v[i] + w[i]$

เพื่อให้ขนาดปัญหา n และจำนวนโพรเซสเซอร์ p เปลี่ยนแปลงได้อย่างอิสระต่อกัน เราจึงขยายโปรแกรมให้ทั่วไปขึ้นดังแสดงในอัลกอริทึม 1 บรรทัดที่ 4 ทำการบวกพร้อมกัน p ครั้งและเขียนสมาชิกของผลลัพธ์ที่อยู่ติดกัน p ตัวลงใน z ลูป for ใช้ทำขั้นตอนขนานพื้นฐานนี้กับ z ทีละส่วนต่อเนื่องกัน ส่วนละ p ตัว เงื่อนไขในบรรทัดที่ 3 รับประกันว่าขั้นขนานขั้นสุดท้ายจะทำการดำเนินการในจำนวนที่ถูกต้อง ในกรณีที่ p หาร n ไม่ลงตัว

อัลกอริทึม 1 (ผลบวกเวกเตอร์บน PRAM)
ข้อมูลเข้า: เวกเตอร์ $v[1..n]$ และ $w[1..n]$ ในหน่วยความจำร่วม
ผลลัพธ์: เวกเตอร์ $z[1..n]$ ในหน่วยความจำร่วม

```
1    local integer h
2    for h = 1 to ⌈n/p⌉ do
3       if (h − 1)p + i ⩽ n then
4          z[(h − 1)p + i] ← v[(h − 1)p + i] + w[(h − 1)p + i]
5       endif
6    enddo
```

เพื่อให้การนำเสนอโปรแกรม PRAM ง่ายขึ้น เราสมมติว่าโพรเซสเซอร์แต่ละตัวมีหน่วยความจำเฉพาะที่ (local memory) อยู่บ้าง หรือกล่าวอย่างเทียบเท่ากันคือมีส่วนหนึ่งของหน่วยความจำร่วมที่เป็นของตนเองโดยเฉพาะ ซึ่งใช้เก็บตัวแปรส่วนตัวของโพรเซสเซอร์ เช่น h และ i โดยทั่วไปเราจะสมมติด้วยว่าพารามิเตอร์อย่าง n และ p อยู่ในหน่วยความจำนี้เช่นกัน ภายใต้สมมติฐานนี้ การอ้างอิงหน่วยความจำร่วมทั้งหมดในอัลกอริทึม 1 เป็นแบบเอกสิทธิ์ และอัลกอริทึมนี้ต้องการเพียง EREW PRAM อัลกอริทึม 1 ใช้จำนวนขั้นในการประมวลผลในอันดับ $\lceil n/p \rceil$ ขั้น ดังนั้นเวลาทำงานแบบขนาน (concurrent running time) คือ $T_C(n,p) = O(n/p)$ □

2 กระบวนทัศน์งาน–เวลา (Work-Time)

แบบจำลอง PRAM แบบเปลือย ๆ นั้นอยู่ในระดับต่ำและใช้งานยุ่งยาก การเขียนอะไรก็ตามที่มากกว่าอัลกอริทึมง่าย ๆ ในแบบจำลองนี้เป็นเรื่องน่าปวดหัว ดังนั้นเราจะเปลี่ยนไปใช้นามธรรมระดับสูงกว่าที่เทียบเท่ากัน เรียกว่ากระบวนทัศน์งาน–เวลา (Work-Time paradigm: WT) เพื่อไม่ต้องกังวลกับรายละเอียดเหล่านี้ หลังจากอธิบายกรอบความคิดนี้แล้ว เราจะนำเสนอทฤษฎีบทของเบรนต์ (Brent's Theorem) ซึ่งช่วยให้แปลงอัลกอริทึม WT เป็นอัลกอริทึมบน PRAM ได้

ในแบบจำลอง PRAM อัลกอริทึมถูกนำเสนอเป็นโปรแกรมที่โพรเซสเซอร์ทุกตัวประมวลผล ในแต่ละขั้น โพรเซสเซอร์ที่ทำงานอยู่ทุกตัวจะดำเนินการหนึ่งการดำเนินการพร้อมกัน ส่วนในแบบจำลอง WT แต่ละขั้นอาจมีการดำเนินการที่ทำพร้อมกันกี่การดำเนินการก็ได้ และไม่ต้องระบุอย่างชัดแจ้งว่าจะจัดตารางการดำเนินการเหล่านี้ลงบนโพรเซสเซอร์อย่างไร ในสัญกรณ์อัลกอริทึมของเรา เราจะใช้โครงสร้าง forall แทนการดำเนินการพร้อมกันเช่นนี้ และจะไม่กล่าวถึงรหัสโพรเซสเซอร์ i และจำนวนโพรเซสเซอร์ p อย่างชัดแจ้งอีก อันที่จริง โครงสร้าง forall เป็นโครงสร้างเดียวที่ทำให้อัลกอริทึม WT แตกต่างจากอัลกอริทึมแบบลำดับ

เรากำหนดมาตรวัดความซับซ้อนสองตัวซึ่งมีขนาดปัญหา n เป็นพารามิเตอร์ให้กับคำอธิบาย WT ของอัลกอริทึม *ความซับซ้อนเชิงงาน* (work complexity) ของอัลกอริทึม เขียนแทนด้วย $W(n)$ คือจำนวนการดำเนินการทั้งหมดที่อัลกอริทึมทำ ในแต่ละขั้นจะมีการดำเนินการหนึ่งหรือหลายการดำเนินการเกิดขึ้นพร้อมกัน *ความซับซ้อนเชิงขั้น* (step complexity) ของอัลกอริทึม เขียนแทนด้วย $S(n)$ คือจำนวนขั้นที่อัลกอริทึมประมวลผล ถ้า $W_i(n)$ คือจำนวนการดำเนินการที่เกิดพร้อมกันในขั้นขนานที่ i แล้ว

$$W(n) = \sum_{i=1}^{S(n)} W_i(n). \tag{1}$$

เมื่อมีสัญกรณ์และนิยามเหล่านี้แล้ว ให้เราพิจารณาอัลกอริทึมแบบขนานตัวที่สองของเรา กำหนดลำดับ a ที่มีสมาชิก $n = 2^k$ ตัวชนิด T ในหน่วยความจำร่วม และตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่ (binary associative operator) $\oplus : T \times T \to T$ สมบัติการเปลี่ยนหมู่หมายความว่า $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ สำหรับทุกสมาชิก a, b และ c ตัวอย่างของตัวดำเนินการเช่นนี้บนชนิดข้อมูลพื้นฐาน ได้แก่ การบวก การคูณ ค่ามากสุด ค่าน้อยสุด AND แบบบูลีน OR แบบบูลีน และการต่อสายอักขระ ตัวดำเนินการที่ซับซ้อนกว่านี้สร้างได้สำหรับชนิดข้อมูลแบบมีโครงสร้างและแบบเวียนเกิด เราต้องการคำนวณปริมาณ $S = \bigoplus_{i=1}^{n} a_i \equiv a_1 \oplus \cdots \oplus a_n$ โดยเก็บผลไว้ในหน่วยความจำร่วมเช่นกัน การดำเนินการนี้เรียกอีกชื่อหนึ่งว่า *การลดรูป* (reduction)

อัลกอริทึม 2 (การลดรูปลำดับ คำอธิบายแบบ WT)
ข้อมูลเข้า: ลำดับ a ที่มีสมาชิก $n = 2^k$ ตัวชนิด T และตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่ $\oplus : T \times T \to T$
ผลลัพธ์: $S = \bigoplus_{i=1}^{n} a_i$

```
   T REDUCE(sequence⟨T⟩ a, ⊕ : T × T → T)
1      T B[1..n]
2      forall i ∈ 1 : n do
3          B[i] ← a_i
4      enddo
5      for h = 1 to k do
6          forall i ∈ 1 : n/2^h do
7              B[i] ← B[2i − 1] ⊕ B[2i]
8          enddo
9      enddo
10     S ← B[1]
11     return S
```

โปรแกรม WT ข้างต้นเป็นคำอธิบายอัลกอริทึมในระดับสูง ไม่มีการอ้างถึงรหัสโพรเซสเซอร์เลย สังเกตด้วยว่าโปรแกรมนี้มีทั้งการดำเนินการแบบลำดับและแบบพร้อมกัน โดยเฉพาะอย่างยิ่ง การกำหนดค่าครั้งสุดท้ายในบรรทัดที่ 10 ทำโดยโพรเซสเซอร์ตัวเดียว (เพราะไม่ได้อยู่ในโครงสร้าง forall) และลูปในบรรทัดที่ 5 เป็นลูป for แบบลำดับ อัลกอริทึมนี้มีรายละเอียดปลีกย่อยสองข้อที่ควรเน้น ข้อแรก ในบรรทัดที่ 7 นิพจน์ทางขวามือทุกตัวต้องถูกประเมินค่าให้เสร็จก่อนที่จะมีการกำหนดค่าใด ๆ เกิดขึ้น ข้อสอง การบวกเกิดขึ้นในลำดับที่ต่างจากใน

โปรแกรมแบบลำดับ (ลองตรวจสอบดู) สมมติฐานว่าการบวกมีสมบัติการเปลี่ยนหมู่จึงสำคัญยิ่งต่อการรับประกันความถูกต้องของผลลัพธ์

ให้เราหา $S(n)$ และ $W(n)$ ของอัลกอริทึม 2 ทั้งสองค่าหาได้แบบอุปนัยจากโครงสร้างของโปรแกรม ในที่นี้ ตัวห้อยหมายถึงหมายเลขบรรทัดในโปรแกรม

$$\begin{aligned} S_{2\text{-}4}(n) &= \Theta(1) & W_{2\text{-}4}(n) &= \Theta(n) \\ S_{6\text{-}8}(n) &= \Theta(1) & W_{6\text{-}8}(n,h) &= \Theta(\tfrac{n}{2^h}) \\ S_{5\text{-}9}(n) = kS_{6\text{-}8}(n) &= \Theta(\lg n) & W_{5\text{-}9}(n) = \textstyle\sum_{h=1}^{k} W_{6\text{-}8}(n,h) &= \Theta(n) \\ S_{10}(n) &= \Theta(1) & W_{10}(n) &= \Theta(1) \\ S(n) = S_{2\text{-}4}(n) + S_{5\text{-}9}(n) + S_{10}(n) &= \Theta(\lg n) & W(n) = W_{2\text{-}4}(n) + W_{5\text{-}9}(n) + W_{10}(n) &= \Theta(n) \end{aligned}$$

เป็นเรื่องน่าอุ่นใจที่ปริมาณงานทั้งหมดของอัลกอริทึมแบบขนานนี้ (เชิงเส้นกำกับ) เท่ากับปริมาณงานของอัลกอริทึมแบบลำดับที่ดีที่สุด ประโยชน์ของความขนานคือจำนวนขั้นที่ลดลง

เราขยายการจำแนกประเภท PRAM ตามการอ้างอิงหน่วยความจำพร้อมกันมาใช้กับแบบจำลอง WT ด้วย อัลกอริทึมข้างต้นมีเพียงการอ่านและการเขียนหน่วยความจำร่วมแบบเอกสิทธิ์ จึงต้องการเพียงแบบจำลองการประมวลผล EREW

Claude อธิบายเพิ่ม: งานกับจำนวนขั้น

ลองนึกภาพการแข่งขันแบบแพ้คัดออก: นักกีฬา $n = 8$ คนแข่งรอบแรก 4 คู่พร้อมกัน รอบสอง 2 คู่ รอบสาม 1 คู่ **จำนวนรอบ** คือ $\lg 8 = 3$ ซึ่งก็คือ $S(n)$ ส่วน**จำนวนการแข่งทั้งหมด** คือ $4 + 2 + 1 = 7 = n - 1$ ซึ่งก็คือ $W(n)$ อัลกอริทึม 2 ทำงานแบบเดียวกันทุกประการ โดย $\oplus$ แทน “การแข่ง” หนึ่งครั้ง

ในเอกสาร *Introduction to Parallel Algorithms* ปริมาณ $S(n)$ นี้ถูกเรียกว่า *สแปน* (span) หรือ *ความลึก* (depth) ซึ่งเป็นแนวคิดเดียวกัน

2.1 ทฤษฎีบทของเบรนต์

ทฤษฎีบทต่อไปนี้ซึ่งเป็นผลงานของเบรนต์ (Brent) เชื่อมโยงความซับซ้อนเชิงงานและเชิงเวลาของอัลกอริทึมแบบขนานที่อธิบายด้วยรูปแบบ WT เข้ากับเวลาทำงานบน PRAM ที่มีโพรเซสเซอร์ p ตัว

ทฤษฎีบท 1 (Brent 1974). *อัลกอริทึม WT ที่มีความซับซ้อนเชิงขั้น $S(n)$ และความซับซ้อนเชิงงาน $W(n)$ สามารถจำลองบน PRAM ที่มีโพรเซสเซอร์ p ตัวได้ในจำนวนขั้นขนานไม่เกิน $\lfloor \frac{W(n)}{p} \rfloor + S(n)$ ขั้น*

พิสูจน์. สำหรับขั้นเวลาแต่ละขั้น i เมื่อ $1 \leqslant i \leqslant S(n)$ ให้ $W_i(n)$ เป็นจำนวนการดำเนินการในขั้นนั้น เราจำลองแต่ละขั้นของอัลกอริทึม WT บน PRAM ที่มีโพรเซสเซอร์ p ตัวได้ใน $\lceil \frac{W_i(n)}{p} \rceil$ ขั้นขนาน โดยจัดตารางการดำเนินการ $W_i(n)$ การดำเนินการลงบนโพรเซสเซอร์ p ตัวเป็นกลุ่ม กลุ่มละ p การดำเนินการ กลุ่มสุดท้ายอาจมีการดำเนินการไม่ครบ p ถ้า p หาร $W_i(n)$ ไม่ลงตัว ในกรณีนี้ เราจัดการดำเนินการที่เหลือให้กับโพรเซสเซอร์ที่มีดัชนีน้อยที่สุด ด้วยกลยุทธ์การจำลองนี้ เวลาที่ใช้จำลองขั้นที่ i[แก้ 1] ของอัลกอริทึม WT จะเป็น $\lceil \frac{W_i(n)}{p} \rceil$ และเวลาทั้งหมดที่ PRAM ที่มีโพรเซสเซอร์ p ตัวใช้จำลองอัลกอริทึมคือ

$$\sum_{i=1}^{S(n)} \left\lceil \frac{W_i(n)}{p} \right\rceil \leqslant \sum_{i=1}^{S(n)} \left(\left\lfloor \frac{W_i(n)}{p} \right\rfloor + 1 \right) \leqslant \left\lfloor \frac{W(n)}{p} \right\rfloor + S(n).$$

ภาพร่างกลยุทธ์การจำลองอย่างง่ายของเรายังไม่ได้จัดการกับความยุ่งยากอีกหลายประการ ตัวอย่างเช่น เพื่อรักษาความหมาย (semantics) ของโครงสร้าง forall โดยทั่วไปเราไม่ควรปรับค่าสมาชิกใด ๆ ทางซ้ายมือของการกำหนดค่าแบบ WT จนกว่าจะประเมินค่าทั้งหมดของนิพจน์ทางขวามือเสร็จแล้ว ซึ่งทำได้โดยใช้ที่เก็บผลลัพธ์ชั่วคราวแล้วคัดลอกไปยังด้านซ้ายมือภายหลัง □

ให้เรากลับไปดูตัวอย่างการลดรูปลำดับอีกครั้ง และลองเขียนอัลกอริทึม PRAM แบบเปลือยสำหรับ PRAM ที่มีโพรเซสเซอร์ p ตัว ตามกลยุทธ์การจำลองที่อธิบายไว้ โครงสร้าง forall แต่ละตัวของอัลกอริทึม 2 ถูกจำลองด้วยลูป for แบบลำดับที่ตัวลูปทำการดำเนินการของตัว forall ได้ครั้งละไม่เกิน p การดำเนินการ

อัลกอริทึม 3 (การลดรูปลำดับ คำอธิบายแบบ PRAM)
ข้อมูลเข้า: ลำดับ a ที่มีสมาชิก $n = 2^k$ ตัวชนิด T ตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่ $\oplus : T \times T \to T$ และรหัสโพรเซสเซอร์ i
ผลลัพธ์: $S = \bigoplus_{i=1}^{n} a_i$

T PRAM-Reduce(sequence$\langle T \rangle$ a, $\oplus : T \times T \to T$)

```
1   T B[1..n]
2   local integer h, j, ℓ
3   for ℓ = 1 to ⌈n/p⌉ do
4     if (ℓ − 1)p + i ⩽ n then
5       B[(ℓ − 1)p + i] ← a_{(ℓ−1)p+i}
6     endif
7   enddo
8   for h = 1 to k do
9     for ℓ = 1 to ⌈(n/2^h)/p⌉ do
10      j ← (ℓ − 1)p + i
11      if j ⩽ n/2^h then
12        B[j] ← B[2j − 1] ⊕ B[2j]
13      endif
14    enddo
15  enddo
16  if i = 1 then
17    S ← B[1]
18  endif
19  return S
```

เวลาทำงานแบบขนานของอัลกอริทึม 3 วิเคราะห์ได้โดยนับจำนวนครั้งที่ตัวลูปถูกประมวลผล

$$T_C(n,p) = \left\lceil \frac{n}{p} \right\rceil \Theta(1) + \sum_{h=1}^{k} \left\lceil \frac{n/2^h}{p} \right\rceil \Theta(1) + \Theta(1) = O\left(\frac{n}{p} + \lg n\right)$$

นี่คือขอบเขตที่ได้จากทฤษฎีบทของเบรนต์สำหรับการจำลองอัลกอริทึม 2 ด้วย PRAM ที่มีโพรเซสเซอร์ p ตัว เพื่อตรวจสอบว่าขอบเขตนี้แน่น (tight) ให้พิจารณาผลรวมข้างต้นในกรณีที่ $p > n$ หรือกรณีที่ p เป็นจำนวนคี่ ภายใต้สมมติฐานเล็กน้อยบางประการ การจำลองนี้รักษาแบบจำลองการเข้าถึงหน่วยความจำร่วมไว้ได้ ตัวอย่างเช่น อัลกอริทึม EREW ในกรอบ WT สามารถจำลองได้ด้วย EREW PRAM

Claude อธิบายเพิ่ม: ทฤษฎีบทของเบรนต์ด้วยตัวเลข

อัลกอริทึม 2 กับ $n = 1024$ มี $W \approx 2n = 2048$ และ $S \approx \lg n + 2 = 12$

- ถ้า $p = 8$: ได้ไม่เกิน $2048/8 + 12 = 268$ ขั้น ส่วน W/p เป็นตัวหลัก จึงเร็วขึ้นเกือบ 8 เท่าเต็ม
- ถ้า $p = 1024$: ได้ไม่เกิน $2 + 12 = 14$ ขั้น ส่วน S เป็นตัวหลัก การเพิ่มโพรเซสเซอร์อีกแทบไม่ช่วยแล้ว

ใจความคือ W/p **เป็นต้นทุนของการแบ่งงาน ส่วน** S **เป็นต้นทุนที่ลดไม่ได้ของการรอกัน** ไม่ว่าจะมีโพรเซสเซอร์มากเท่าไร

2.2 การออกแบบอัลกอริทึมแบบขนานที่ดี

อัลกอริทึมบน PRAM มีความซับซ้อนเชิงเวลาที่มีทั้งขนาดปัญหาและจำนวนโพรเซสเซอร์เป็นพารามิเตอร์ กำหนดอัลกอริทึมบน PRAM ที่มีเวลาทำงาน $T_C(n,p)$ ให้ $T_S(n)$ เป็นความซับซ้อนเชิงเวลาแบบลำดับที่เหมาะที่สุด (หรือดีที่สุดเท่าที่รู้) ของปัญหานั้น เรานิยาม*อัตราเร่ง*

(speedup)

$$\mathcal{SP}(n,p) = \frac{T_S(n)}{T_C(n,p)} \tag{2}$$

ว่าเป็นตัวคูณของการปรับปรุงเวลาทำงานที่ได้จากการประมวลผลแบบขนาน อัตราเร่งที่ดีที่สุดที่เราหวังได้ (สำหรับอัลกอริทึมแบบเชิงกำหนด: deterministic) คือ $\Theta(p)$ เมื่อใช้โพรเซสเซอร์ p ตัว อัตราเร่งที่มากกว่านี้เชิงเส้นกำกับจะขัดแย้งกับสมมติฐานที่ว่าความซับซ้อนเชิงเวลาแบบลำดับของเราเหมาะที่สุดแล้ว เพราะเราจะสร้างอัลกอริทึมแบบลำดับที่เร็วกว่าได้ด้วยการจำลองอัลกอริทึม PRAM ของเราแบบลำดับ

อัลกอริทึมแบบขนานในกรอบ WT มีลักษณะเฉพาะด้วยมาตรวัดความซับซ้อนเชิงขั้นและเชิงงานซึ่งมีพารามิเตอร์ตัวเดียว ความซับซ้อนเชิงงาน $W(n)$ เป็นมาตรวัดที่สำคัญที่สุด จากทฤษฎีบทของเบรนต์ เราจำลองอัลกอริทึม WT บน PRAM ที่มีโพรเซสเซอร์ p ตัวได้ในเวลา

$$T_C(n,p) = O\Big(\frac{W(n)}{p} + S(n)\Big). \tag{3}$$

ถ้า $W(n)$ มีอันดับโตกว่า $T_S(n)$ เชิงเส้นกำกับ เราจะเห็นว่าเมื่อจำนวนโพรเซสเซอร์ p คงที่ การเพิ่มขนาดปัญหาจะทำให้อัตราเร่งลดลง กล่าวคือ

$$\lim_{n\to\infty} \mathcal{SP}(n,p) = \lim_{n\to\infty} \frac{T_S(n)}{\lfloor \frac{W(n)}{p} \rfloor + S(n)} = 0$$

เนื่องจากการเพิ่ม p มีขีดจำกัดตายตัวในสถานการณ์จริงหลายสถานการณ์ เราจึงต้องการสร้างอัลกอริทึม WT แบบขนานที่ $W(n) = \Theta(T_S(n))$ อัลกอริทึมเช่นนี้เรียกว่า*มีประสิทธิภาพเชิงงาน* (work-efficient)

เป้าหมายที่สองคือการทำให้ความซับซ้อนเชิงขั้น $S(n)$ น้อยที่สุด จากทฤษฎีบทของเบรนต์ เราจำลองอัลกอริทึม WT ที่มีประสิทธิภาพเชิงงานบน PRAM ที่มีโพรเซสเซอร์ p ตัวได้ในเวลา

$$T_C(n,p) = O\Big(\frac{T_S(n)}{p} + S(n)\Big). \tag{4}$$

ดังนั้น อัตราเร่งที่ได้บน PRAM ที่มีโพรเซสเซอร์ p ตัวคือ

$$\mathcal{SP}(n,p) = \Omega\left(\frac{T_S(n)}{\frac{T_S(n)}{p} + S(n)}\right) = \Omega\Big(\frac{pT_S(n)}{T_S(n) + pS(n)}\Big). \tag{5}$$

ดังนั้น $\mathcal{SP}(n,p)$ จะเป็น $\Theta(p)$ (ดีที่สุดเท่าที่หวังได้) ตราบใดที่

$$p = O\Big(\frac{T_S(n)}{S(n)}\Big). \tag{6}$$

ดังนั้น ในบรรดาอัลกอริทึมแบบขนานที่มีประสิทธิภาพเชิงงานสองตัวสำหรับปัญหาเดียวกัน ตัวที่มีความซับซ้อนเชิงขั้นน้อยกว่าจะขยายขนาดได้ดีกว่า (more scalable) ในแง่ที่รักษาอัตราเร่งที่เหมาะที่สุดไว้ได้ในช่วงจำนวนโพรเซสเซอร์ที่กว้างกว่า

Claude อธิบายเพิ่ม: อ่านสมการ (6) อย่างไร

$T_S(n)/S(n)$ คือ “ความขนาน” (parallelism) ของอัลกอริทึม หรือจำนวนโพรเซสเซอร์สูงสุดที่ยังใช้ได้คุ้มค่า เช่น การลดรูปลำดับมี $T_S = n$ และ $S = \lg n$ จึงใช้โพรเซสเซอร์ได้ถึงราว $n/\lg n$ ตัวโดยอัตราเร่งยังเป็นเชิงเส้น ถ้า $n = 2^{20} \approx 10^6$ ก็ใช้ได้ถึงประมาณ 5×10^4 ตัว

3 เทคนิคพื้นฐานในการออกแบบอัลกอริทึมบน PRAM

ต่อไปเราจะอภิปรายเทคนิคการออกแบบอัลกอริทึมหลากหลายแบบสำหรับ PRAM ดังที่คุณจะได้เห็น เทคนิคเหล่านี้ใช้กับโครงสร้างข้อมูลได้หลายชนิด และมักมีเทคนิคคู่เทียบในการออกแบบอัลกอริทึมแบบลำดับ

3.1 ต้นไม้สมดุล

เทคนิคการออกแบบอัลกอริทึมบน PRAM ที่ใช้กันทั่วไปเทคนิคหนึ่งคือการสร้างต้นไม้ทวิภาคแบบสมดุล (balanced binary tree) บนข้อมูลเข้า แล้วกวาดผ่านต้นไม้นี้ทั้งขาไปและขากลับจากราก "ต้นไม้" นี้ไม่ใช่โครงสร้างข้อมูลจริง แต่เป็นแนวคิดในหัวของเรา ซึ่งมักเกิดขึ้นจริงในรูปของต้นไม้การเวียนเกิด (recursion tree) เราได้เห็นการใช้เทคนิคนี้ไปแล้วในตัวอย่างการหาผลรวมของอาร์เรย์ เทคนิคนี้ใช้ได้อย่างกว้างขวาง ใช้สร้างอัลกอริทึมแบบขนานที่มีประสิทธิภาพเชิงงานสำหรับปัญหาต่าง ๆ เช่น ผลบวกสะสม (prefix sum) การกระจายค่า (broadcast) และการบีบอัดอาร์เรย์ (array compaction) เราจะดูกรณีผลบวกสะสม

ในปัญหาผลบวกสะสม (เรียกอีกชื่อว่า *parallel prefix* หรือ *scan*) เราได้รับลำดับข้อมูลเข้า $x = \langle x_1, \ldots, x_n \rangle$ ซึ่งมีสมาชิกชนิด T และตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่ $\oplus : T \times T \to T$ ผลลัพธ์ที่ต้องสร้างคือลำดับ $s = \langle s_1, \ldots, s_n \rangle$ โดยที่สำหรับ $1 \leqslant k \leqslant n$ เราต้องการให้ $s_k = \bigoplus_{i=1}^{k} x_i$

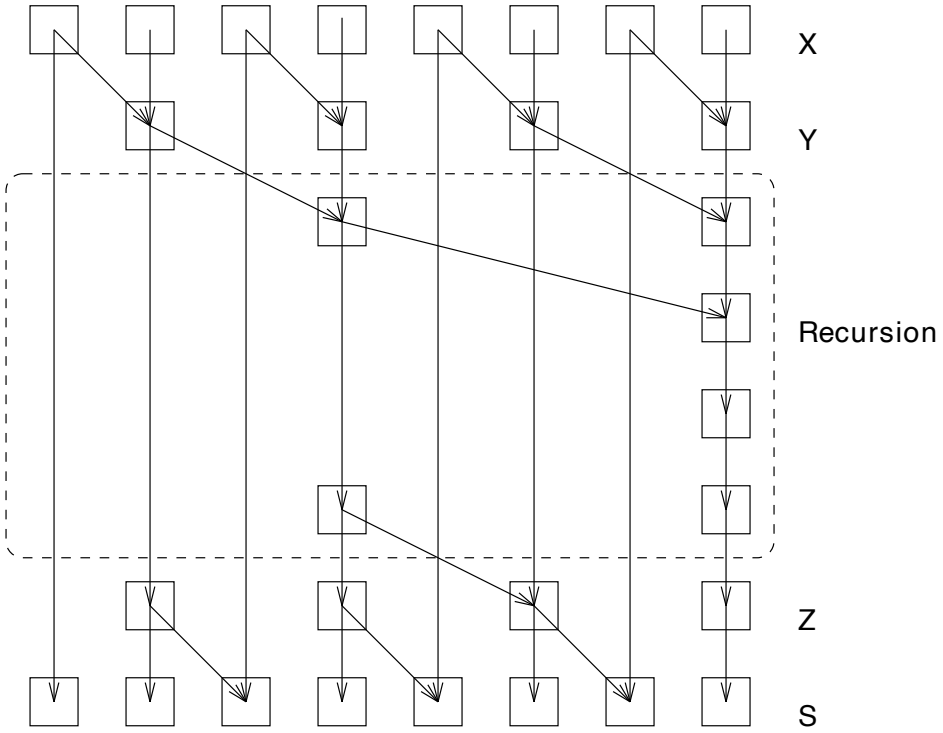
ความซับซ้อนเชิงเวลาแบบลำดับ $T_S(n)$ ของปัญหานี้คือ $\Theta(n)$ อย่างชัดเจน ขอบเขตล่างตามมาโดยตรงจากข้อเท็จจริงที่ต้องเขียนสมาชิกผลลัพธ์ n ตัว และขอบเขตบนได้จากอัลกอริทึมที่คำนวณ s_{i+1} เป็น $s_i \oplus x_{i+1}$ ดังนั้นเป้าหมายของเราคือสร้างอัลกอริทึมแบบขนานที่มีความซับซ้อนเชิงงาน $\Theta(n)$ เราจะทำได้โดยใช้เทคนิคต้นไม้สมดุล อัลกอริทึม WT ของเราจะต่างจากอัลกอริทึมก่อนหน้าตรงที่เป็นแบบเวียนเกิด (recursive) ตามปกติ เราจะสมมติว่า $n = 2^k$ เพื่อให้การนำเสนอง่ายขึ้น

อัลกอริทึม 4 (ผลบวกสะสม)
ข้อมูลเข้า: ลำดับ x ที่มีสมาชิก $n = 2^k$ ตัวชนิด T และตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่ $\oplus : T \times T \to T$
ผลลัพธ์: ลำดับ s ที่มีสมาชิก $n = 2^k$ ตัวชนิด T โดย $s_k = \bigoplus_{i=1}^{k} x_i$ สำหรับ $1 \leqslant k \leqslant n$

```
    sequence⟨T⟩ Scan(sequence⟨T⟩ x, ⊕ : T × T → T)
1       if n = 1 then
2           s_1 ← x_1
3           return s
4       endif
5       forall i ∈ 1 : n/2 do
6           y_i ← x_{2i−1} ⊕ x_{2i}
7       enddo
8       ⟨z_1, ..., z_{n/2}⟩ ← Scan(⟨y_1, ..., y_{n/2}⟩, ⊕)
9       forall i ∈ 1 : n do
10          if even(i) then
11              s_i ← z_{i/2}
12          elsif i = 1 then
13              s_1 ← x_1
14          else
15              s_i ← z_{(i−1)/2} ⊕ x_i
16          endif
17      enddo
18      return s
```

รูปที่ 1 แสดงการไหลของข้อมูลในอัลกอริทึม 4

ทฤษฎีบท 2. *อัลกอริทึม 4 คำนวณผลบวกสะสมของลำดับ x ได้ถูกต้อง โดยมีความซับซ้อนเชิงขั้น $\Theta(\lg n)$ และความซับซ้อนเชิงงาน $\Theta(n)$*



รูปที่ 1: การไหลของข้อมูลในอัลกอริทึม 4 สำหรับข้อมูลเข้าตัวอย่างที่มีสมาชิกแปดตัว (Recursion = ส่วนที่เรียกตัวเองซ้ำ)

พิสูจน์. ความถูกต้องของอัลกอริทึมพิสูจน์ได้ด้วยการอุปนัยอย่างง่ายบน $k = \lg n$ กรณีฐานคือ $k = 0$ ซึ่งถูกต้องตามบรรทัดที่ 2 ต่อไปสมมติว่าอัลกอริทึมถูกต้องสำหรับข้อมูลเข้าขนาด 2^k และพิจารณาข้อมูลเข้าขนาด 2^{k+1} จากสมมติฐานการอุปนัย $z = \textsc{Scan}(y, \oplus)$ ที่คำนวณในบรรทัดที่ 8 ถูกต้อง ดังนั้น $z_i = y_1 \oplus \cdots \oplus y_i = x_1 \oplus \cdots \oplus x_{2i}$ ต่อไปพิจารณาความเป็นไปได้สามกรณีของ s_i ถ้า $i = 2j$ เป็นจำนวนคู่ (บรรทัดที่ 11) แล้ว $s_i = z_j = x_1 \oplus \cdots \oplus x_{2j} = x_1 \oplus \cdots \oplus x_i$ ถ้า $i = 1$ (บรรทัดที่ 13) แล้ว $s_1 = x_1$ สุดท้าย ถ้า $i = 2j + 1$ เป็นจำนวนคี่ (บรรทัดที่ 15) แล้ว $s_i = z_j \oplus x_i = x_1 \oplus \cdots \oplus x_{i-1} \oplus x_i$ สามกรณีนี้ครอบคลุมทุกกรณี จึงสรุปได้ว่าอัลกอริทึมถูกต้อง

สำหรับขอบเขตของทรัพยากร เราสังเกตว่าความซับซ้อนเชิงขั้นและเชิงงานสอดคล้องกับความสัมพันธ์เวียนเกิด (recurrence) ต่อไปนี้

$$S(n) = S(n/2) + \Theta(1) \tag{7}$$

$$W(n) = W(n/2) + \Theta(n) \tag{8}$$

ความสัมพันธ์เวียนเกิดมาตรฐานเหล่านี้มีผลเฉลยเป็น $S(n) = \Theta(\lg n)$ และ $W(n) = \Theta(n)$ □

ดังนั้นเราจึงได้อัลกอริทึมแบบขนานที่มีประสิทธิภาพเชิงงานซึ่งทำงานบน EREW PRAM ได้ และรักษาอัตราเร่งที่เหมาะที่สุดได้เมื่อใช้โพรเซสเซอร์ $O(\frac{n}{\lg n})$ ตัว

ทำไมแบบจำลอง PRAM ที่น้อยที่สุดจึงเป็น EREW ทั้งที่ดูเหมือนมีการอ่านค่าใน $z[1 : n/2 - 1]$ พร้อมกัน (ดังที่รูปที่ 1 ชี้ให้เห็น) จริงอยู่ที่ค่าแต่ละค่าเหล่านี้ต้องถูกอ่านสองครั้ง แต่การอ่านเหล่านี้สามารถทำให้เป็นลำดับ (serialize) ได้โดยไม่เปลี่ยนความซับซ้อนเชิงเส้นกำกับของอัลกอริทึม อันที่จริง เนื่องจากการอ่านเกิดขึ้นในแขนงต่างกันของเงื่อนไข (บรรทัดที่ 11 และ 15) การอ่านจึงถูกทำให้เป็นลำดับโดยอัตโนมัติขณะประมวลผลภายใต้แบบจำลอง PRAM แบบประสานจังหวะ ในหัวข้อถัดไป เราจะได้เห็นตัวอย่างที่โพรเซสเซอร์มากกว่าจำนวนคงที่พยายามอ่านค่าเดียวกัน ทำให้แบบจำลอง PRAM ที่น้อยที่สุดเป็น CREW

เรานิยามการดำเนินการ scan ได้สองรูปแบบ คือแบบรวมตัวเอง (inclusive) (ดังข้างต้น) และแบบไม่รวมตัวเอง (exclusive) สำหรับ exclusive scan เราต้องการให้ตัวดำเนินการ $\oplus$ มีสมาชิกเอกลักษณ์ (identity element) I (หมายความว่า $x \oplus I = I \oplus x = x$ สำหรับทุกสมาชิก x) แล้ว exclusive scan นิยามดังนี้: ถ้า t เป็นลำดับผลลัพธ์ แล้ว $t_1 = I$ และ $t_k = x_1 \oplus \cdots \oplus x_{k-1}$ สำหรับ

$1 < k \leqslant n$ เห็นได้ชัดว่าเราได้ inclusive scan จาก exclusive scan ด้วยความสัมพันธ์ $s_k = t_k \oplus x_k$ ส่วนในทิศกลับ สังเกตว่าสำหรับ $k > 1$ จะได้ $t_k = s_{k-1}$ และ $t_1 = I$

สุดท้าย ถ้า $n \neq 2^k$ จะทำอย่างไร ถ้า $2^k < n < 2^{k+1}$ เราก็เติมข้อมูลเข้าให้มีขนาด 2^{k+1} (pad) ใช้อัลกอริทึม 4 แล้วทิ้งค่าส่วนเกิน เนื่องจากวิธีนี้เพิ่มขนาดปัญหาไม่เกินสองเท่า เราจึงยังคงความซับซ้อนเชิงเส้นกำกับไว้ได้

Claude อธิบายเพิ่ม: ไล่ scan ด้วยมือ

ให้ $x = \langle 3, 1, 4, 1, 5, 9, 2, 6 \rangle$ และ $\oplus = +$

- บรรทัด 5–7: จับคู่บวกกันได้ $y = \langle 4, 5, 14, 8 \rangle$
- บรรทัด 8: scan ของ y แบบเวียนเกิดได้ $z = \langle 4, 9, 23, 31 \rangle$ ซึ่งคือผลบวกสะสมที่ตำแหน่งคู่ ของ x
- บรรทัด 9–17: ตำแหน่งคู่ลอกจาก z ตรง ๆ ส่วนตำแหน่งคี่ใช้ z ตัวก่อนหน้าบวก x_i ได้ $s = \langle 3, 4, 8, 9, 14, 23, 25, 31 \rangle$

ทุกระดับของการเวียนเกิดทำงานครึ่งหนึ่งของระดับก่อน รวมงานจึงเป็น $n + n/2 + n/4 + \cdots \leq 2n$ นี่คือเหตุผลที่ได้ $W(n) = \Theta(n)$

3.2 การกระโดดตัวชี้

เทคนิคการกระโดดตัวชี้ (pointer jumping) หรือการเพิ่มตัวชี้เป็นสองเท่า (pointer doubling) ช่วยให้ประมวลผลโครงสร้างข้อมูลแบบเชื่อมโยง (linked data structures) เช่น ลิสต์และต้นไม้ แบบขนานได้อย่างรวดเร็ว โดยทั่วไปเราจะวาดต้นไม้ให้มีเส้นเชื่อมชี้จากลูกไปยังพ่อแม่ (เช่นเดียวกับที่ทำเมื่อแทนเซตไม่มีส่วนร่วม: disjoint sets) จำได้ว่าต้นไม้มีทิศทางแบบมีราก (rooted directed tree) T คือกราฟมีทิศทางที่มีจุดยอดรากพิเศษ r ซึ่งดีกรีออก (outdegree) ของรากเป็นศูนย์ ขณะที่ดีกรีออกของจุดยอดอื่นทั้งหมดเป็นหนึ่ง และมีเส้นทางมีทิศทางจากจุดยอดที่ไม่ใช่รากทุกจุดไปยังราก ปัญหาตัวอย่างของเราคือการหารากทั้งหมดของป่า (forest) ของต้นไม้มีทิศทาง ซึ่งมีจุดยอดรวม n จุด (และเส้นเชื่อมไม่เกิน $n - 1$ เส้น)

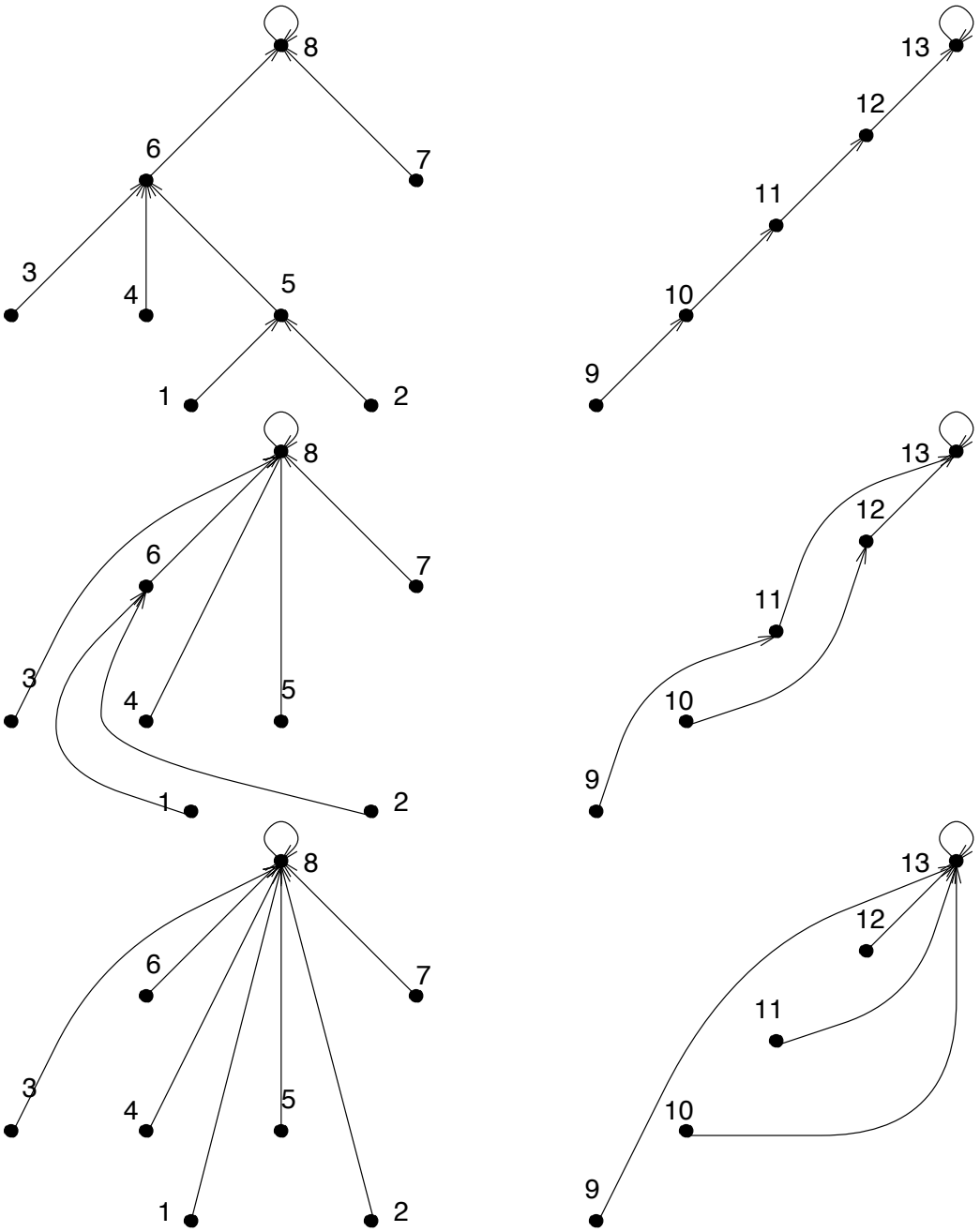
เราจะแทนป่าด้วยอาร์เรย์ P (มาจาก “Parent”) ของจำนวนเต็ม n ตัว โดย $P[i] = j$ ก็ต่อเมื่อ (i, j) เป็นเส้นเชื่อมในป่า เราจะใช้วงวนตนเอง (self-loop) เพื่อบ่งบอกราก กล่าวคือ จุดยอด i เป็นรากก็ต่อเมื่อ $P[i] = i$ ผลลัพธ์ที่ต้องการคืออาร์เรย์ S ซึ่ง $S[j]$ คือรากของต้นไม้ที่มีจุดยอด j สำหรับ $1 \leqslant j \leqslant n$ อัลกอริทึมแบบลำดับที่ใช้การค้นหาแนวลึก (depth-first search) ให้ $T_S(n) = O(n)$

อัลกอริทึม 5 (รากของป่า)
ข้อมูลเข้าป่าที่มี n จุดยอด แทนด้วยอาร์เรย์พ่อแม่ $P[1..n]$
ผลลัพธ์: อาร์เรย์ $S[1..n]$ ที่ให้รากของต้นไม้ที่มีจุดยอดแต่ละจุด

```
1    forall i ∈ 1 : n do
2        S[i] ← P[i]
3        while S[i] ≠ S[S[i]] do
4            S[i] ← S[S[i]]
5        endwhile
6    enddo
```

ขอย้ำอีกครั้งว่าในบรรทัดที่ 4 ค่า $S[S[i]]$ ทุกตัวจะถูกประเมินก่อนที่จะมีการกำหนดค่าให้ $S[i]$ ใด ๆ ตัวชี้ $S[i]$ คือ “ตัวตามหลัง” (successor) ปัจจุบันของจุดยอด i และในตอนเริ่มต้นคือพ่อแม่ของมัน ในแต่ละขั้น ระยะทางในต้นไม้ระหว่างจุดยอด i กับ $S[i]$ จะเพิ่มเป็นสองเท่า ตราบใดที่ $S[S[i]]$ ยังไม่ใช่รากของป่า ให้ h เป็นความสูงมากที่สุดของต้นไม้ในป่า ความถูกต้องของอัลกอริทึมพิสูจน์ได้ด้วยการอุปนัยบน h อัลกอริทึมนี้ทำงานบน CREW PRAM การเขียนทั้งหมดไม่ซ้ำตำแหน่งกัน แต่โพรเซสเซอร์มากกว่าจำนวนคงที่อาจอ่านค่าจากจุดยอดร่วมเดียวกัน ดังแสดงในรูปที่ 2 สำหรับความซับซ้อนเชิงขั้นและเชิงงานของอัลกอริทึม เราสังเกตว่าลูป while วนซ้ำ $\Theta(\lg h)$ ครั้ง และแต่ละรอบใช้ $\Theta(1)$ ขั้นและงาน $O(n)$ ดังนั้น $S(n) = \Theta(\lg h)$ และ $W(n) = O(n \lg h)$ ขอบเขตเหล่านี้อ่อน แต่เรายืนยันอะไรที่แข็งกว่านี้ไม่ได้หากไม่ตั้งสมมติฐานเพิ่มเกี่ยวกับข้อมูลเข้า อัลกอริทึมนี้ไม่มีประสิทธิภาพเชิงงานเว้นแต่ h เป็นค่าคงที่ โดยเฉพาะสำหรับลิสต์เชื่อมโยง (linked list) อัลกอริทึมใช้ $\Theta(\lg n)$ ขั้นและงาน $\Theta(n \lg n)$ แบบฝึกหัดที่น่าสนใจคือการเก็บค่าระยะทาง d_i จาก

จุดยอด i ไปยังตัวตามหลังของมันซึ่งวัดตามเส้นทางในต้นไม้ไว้กับจุดยอดแต่ละจุด แล้วปรับอัลกอริทึมให้รักษาค่านี้ได้ถูกต้อง เมื่ออัลกอริทึมจบ d_i จะเป็นระยะทางจากจุดยอด i ถึงรากของต้นไม้ของมัน



รูปที่ 2: การทำงานสามรอบของบรรทัดที่ 4 ในอัลกอริทึม 5 บนป่าที่มี 13 จุดยอดและต้นไม้สองต้น

อัลกอริทึมนี้ข้ามรายละเอียดสำคัญไปข้อหนึ่ง คือเราจะรู้ได้อย่างไรว่าเมื่อไรควรหยุดวนลูป while แนวคิดแรกคือใช้จำนวนรอบที่ตายตัวดังนี้ เนื่องจากความสูงของต้นไม้ที่สูงที่สุดมีขอบเขตบนชัดเจนคือ n เราจึงไม่จำเป็นต้องวนลูปกระโดดตัวชี้เกิน $\lg n$ ครั้ง

forall $i \in 1 : n$ do
  $S[i] \leftarrow P[i]$
enddo
for $k = 1$ to $\lg n$ do

```
    forall i ∈ 1 : n do
      S[i] ← S[S[i]]
    enddo
  enddo
```

วิธีนี้ถูกต้องแต่ไม่มีประสิทธิภาพ เพราะป่าของเราอาจประกอบด้วยต้นไม้เตี้ยและแผ่กว้างจำนวนมาก ความซับซ้อนเชิงงานของมันคือ $\Theta(n \lg n)$ แทนที่จะเป็น $O(n \lg h)$ และความซับซ้อนเชิงขั้นคือ $\Theta(\lg n)$ แทนที่จะเป็น $O(\lg h)$ แนวคิดที่สองคืออัลกอริทึมตรวจจับการสิ้นสุดแบบ “ซื่อตรง” (honest termination detection) ดังนี้

```
forall i ∈ 1 : n do
  S[i] ← P[i]
enddo
repeat
  forall i ∈ 1 : n do
    S[i] ← S[S[i]]
    M[i] ← if S[i] ≠ S[S[i]] then 1 else 0 endif
  enddo
  t ← Reduce(M, +)
until t = 0
```

วิธีนี้ให้ความซับซ้อนเชิงงาน $O(n \lg h)$ ตามที่ต้องการ แต่ความซับซ้อนเชิงขั้นเป็น $O(\lg n \lg h)$ เพราะเราทำการลดรูปที่ใช้ $O(\lg n)$ ขั้นในแต่ละรอบจากทั้งหมด $\lg h$ รอบ

ในการออกแบบอัลกอริทึมแบบขนาน การทำให้ความซับซ้อนเชิงงานน้อยที่สุดสำคัญที่สุด ดังนั้นเราน่าจะเลือกใช้การตรวจจับการสิ้นสุดแบบซื่อตรงในอัลกอริทึม 5 อย่างไรก็ตาม อัลกอริทึมพื้นฐานนี้แม้จะปรับแล้วก็ยังไม่มีประสิทธิภาพเชิงงานเต็มที่ เราทำให้อัลกอริทึมมีประสิทธิภาพเชิงงานได้โดยใช้เทคนิคที่นำเสนอในหัวข้อถัดไป รายละเอียดดูได้ใน JáJá §3.1

Claude อธิบายเพิ่ม: ทำไมกระโดดตัวชี้จึงเร็ว

บนลิสต์เชื่อมโยง $1 \to 2 \to \cdots \to 16$ (ให้ 16 เป็นราก) หลังรอบที่ 1 ทุกจุดชี้ไปข้างหน้า 2 ช่อง หลังรอบที่ 2 ชี้ไป 4 ช่อง รอบที่ 3 ชี้ 8 ช่อง รอบที่ 4 ทุกจุดชี้ถึงรากแล้ว ระยะที่ “มองเห็น” เพิ่มเป็นสองเท่าทุกรอบ จึงใช้แค่ $\lg 16 = 4$ รอบ แต่ทุกรอบทุกจุดยอดยังทำงานอยู่ จึงเสียงาน $n \lg n$ ซึ่งมากกว่าการเดินลิสต์แบบลำดับที่ใช้แค่ n

3.3 การต่อเรียงอัลกอริทึม

อัลกอริทึมแบบขนานที่มีความซับซ้อนเชิงงานไม่เหมาะที่สุดไม่ควรถูกปัดตกไปเสียทีเดียว *การต่อเรียงอัลกอริทึม* (algorithm cascading) คือการประกอบอัลกอริทึมที่ไม่มีประสิทธิภาพเชิงงาน (แต่เร็ว) เข้ากับอัลกอริทึมที่มีประสิทธิภาพ (แต่ช้ากว่า หรือเป็นแบบลำดับ) เพื่อปรับปรุงประสิทธิภาพเชิงงาน บางครั้งเราแปลงอัลกอริทึมที่ไม่มีประสิทธิภาพเชิงงานให้เป็นอัลกอริทึมที่มีประสิทธิภาพเชิงงานได้ด้วยเทคนิคนี้

ปัญหาตัวอย่างที่ใช้แสดงวิธีการนี้เป็นดังนี้ กำหนดลำดับ $L[1..n]$ ของจำนวนเต็มในช่วง $1 : k$ เมื่อ $k = \lg n$ ให้หาว่าจำนวนเต็มแต่ละตัวในช่วงนี้ปรากฏใน L กี่ครั้ง นั่นคือ ให้คำนวณฮิสโทแกรม $R[1..k]$ ซึ่งสำหรับทุก $1 \leqslant i \leqslant k$ ค่า $R[i]$ บันทึกจำนวนสมาชิกใน L ที่มีค่าเท่ากับ i

อัลกอริทึมแบบลำดับที่เหมาะที่สุดสำหรับปัญหานี้ ซึ่งมี $T_S(n) = \Theta(n)$ เป็นดังนี้

```
R[1 : k] ← 0
for i = 1 to n do
  R[L[i]] ← R[L[i]] + 1
enddo
```

ในการสร้างอัลกอริทึมแบบขนาน เราอาจสร้าง $C[1..n, 1..k]$ แบบขนาน โดยที่

$$C_{i,j} = \begin{cases} 1 & \text{ถ้า } L[i] = j \\ 0 & \text{กรณีอื่น} \end{cases}$$

จากนั้นเพื่อหาจำนวนครั้งที่ j ปรากฏใน L เราก็แค่รวมคอลัมน์ j ของ C นั่นคือ $C[1:n, j]$ อัลกอริทึมฉบับสมบูรณ์คือ

```
forall i ∈ 1:n, j ∈ 1:k do
  C[i, j] ← 0
enddo
forall i ∈ 1:n do
  C[i, L[i]] ← 1
enddo
forall j ∈ 1:k do
  R[j] ← REDUCE(C[1:n, j], +)
enddo
```

ความซับซ้อนเชิงขั้นของอัลกอริทึมนี้คือ $\Theta(\lg n)$ ซึ่งเป็นผลจากความซับซ้อนเชิงขั้นของการดำเนินการ REDUCE ส่วนความซับซ้อนเชิงงานคือ $\Theta(nk) = \Theta(n \lg n)$ ซึ่งเป็นผลจากโครงสร้าง forall ตัวแรกและตัวสุดท้าย อัลกอริทึมนี้ไม่มีประสิทธิภาพเชิงงาน เพราะ C ใหญ่เกินกว่าจะกำหนดค่าเริ่มต้นและรวมผลได้ด้วยงานเพียง $O(n)$

อย่างไรก็ตาม รูปแบบหนึ่งของอัลกอริทึมแบบลำดับที่มีประสิทธิภาพข้างต้นสามารถสร้างและรวมแถวต่อเนื่องกัน k แถวของ C ได้ใน $O(k) = O(\lg n)$ ขั้นและงาน (แบบลำดับ) เมื่อใช้อัลกอริทึมแบบลำดับนี้แบบขนานกัน $m = n/\lg n$ ชุด เราสร้าง $\hat{C}[1..m, 1..k]$ ได้ใน $O(k) = O(\lg n)$ ขั้น โดยทำงานรวม $O(mk) = O(n)$ จากนั้นเราหาผลรวมของแต่ละคอลัมน์ของ $\hat{C}$ ได้ภายใต้ขอบเขตความซับซ้อนเดียวกันนี้

อัลกอริทึม 6 (อัลกอริทึมต่อเรียงที่มีประสิทธิภาพเชิงงานสำหรับปัญหานับป้ายกำกับ)
ข้อมูลเข้า: ลำดับ $L[1..n]$ ที่มีค่าอยู่ในช่วง $1:k$
ผลลัพธ์: ลำดับ $R[1..k]$ ของจำนวนครั้งที่ค่าแต่ละค่าปรากฏใน L

```
1   integer Ĉ[1..m, 1..k]
2   forall i ∈ 1:m, j ∈ 1:k do
3     Ĉ[i, j] ← 0
4   enddo
5   forall i ∈ 1:m do
6     for j = 1 to k do
7       Ĉ[i, L[(i − 1)k + j]] ← Ĉ[i, L[(i − 1)k + j]] + 1
8     enddo
9   enddo
10  forall j ∈ 1:k do
11    R[j] ← REDUCE(Ĉ[1:m, j], +)
12  enddo
```

อัลกอริทึมแบบต่อเรียงนี้มี $S(n) = O(\lg n)$ และ $W(n) = O(n)$ จึงถูกทำให้มีประสิทธิภาพเชิงงานได้โดยไม่ทำให้ความซับซ้อนเชิงขั้นเพิ่มขึ้นเชิงเส้นกำกับ อัลกอริทึมนี้ทำงานบนแบบจำลอง EREW PRAM

Claude อธิบายเพิ่ม: สูตรของการต่อเรียง

แบ่งข้อมูล n ตัวเป็น $n/\lg n$ ก้อน ก้อนละ $\lg n$ ตัว

1. **ภายในก้อน:** ใช้อัลกอริทึม*แบบลำดับ*ที่มีประสิทธิภาพ ทุกก้อนทำพร้อมกัน ใช้เวลา $O(\lg n)$ งานรวม $O(n)$
2. **ระหว่างก้อน:** เหลือข้อมูลเพียง $n/\lg n$ ชิ้น จึงใช้อัลกอริทึม*แบบขนาน*ที่เปลืองงานได้โดยไม่เสียหาย (ในที่นี้คือ k ตารางย่อย คอลัมน์ละ m ตัว รวม $mk = n$)

เทคนิคเดียวกันนี้ทำให้การกระโดดตัวชี้ (หัวข้อ 3.2) และการหาค่ามากสุดบน CRCW (หัวข้อ 5.2) มีประสิทธิภาพเชิงงานได้

3.4 ทัวร์ออยเลอร์

เทคนิคทัวร์ออยเลอร์ (Euler tour) ใช้ในอัลกอริทึมแบบขนานหลายตัวที่ทำงานกับข้อมูลที่มีโครงสร้างเป็นต้นไม้ ชื่อนี้มาจากวงจรออยเลอร์ (Euler circuit) ของกราฟมีทิศทาง จำได้ว่า*วงจรออยเลอร์*ของกราฟมีทิศทางคือวัฏจักรมีทิศทาง (directed cycle) ที่ผ่านเส้นเชื่อมทุกเส้นเพียงครั้งเดียว ถ้าเรานำต้นไม้ $T = (V, E)$ มาสร้างกราฟมีทิศทาง $T' = (V, E')$ โดยแทนเส้นเชื่อม $(u, v) \in E$ แต่ละเส้นด้วยเส้นเชื่อมมีทิศทางสองเส้น (u, v) และ (v, u) ใน E' กราฟ T' จะมีวงจรออยเลอร์ เราเรียกวงจรออยเลอร์ของ T' ว่า*ทัวร์ออยเลอร์*ของ T ในเชิงรูปนัย เราระบุทัวร์ออยเลอร์ของ T โดยนิยามฟังก์ชันตัวตามหลัง (successor function) $s : E' \to E'$ ซึ่ง $s(e)$ คือเส้นเชื่อมที่ตามหลังเส้นเชื่อม e ในทัวร์ ถ้านิยามฟังก์ชันตัวตามหลังอย่างเหมาะสม เราสร้างทัวร์ออยเลอร์ที่ไล่จุดยอดตามการท่องแบบ inorder, preorder หรือ postorder ได้

ตอนนี้ให้พิจารณาต้นไม้มีรากก่อน การแทนต้นไม้ด้วยทัวร์ออยเลอร์มีความเชื่อมโยงอย่างแนบแน่นกับการแทนโครงสร้างต้นไม้ด้วยลำดับวงเล็บที่สมดุล (balanced parenthesis sequence) จำได้ว่าลำดับวงเล็บที่สมดุลทุกลำดับตีความเป็นต้นไม้มีรากได้ การแทนแบบนี้มีสมบัติสำคัญคือ ลำดับย่อยที่แทนต้นไม้ย่อยใด ๆ ก็สมดุลเช่นกัน ผลที่ตามมาจากสมบัตินี้คือ ถ้า $\oplus$ เป็นตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่และมีตัวผกผัน (inverse) แล้วเราวางสมาชิก e ไว้ที่วงเล็บเปิดแต่ละตัว และวางตัวผกผัน $-e$ ไว้ที่วงเล็บปิดแต่ละตัว ผลรวม (เทียบกับ $\oplus$) ของสมาชิกในต้นไม้ย่อยใด ๆ จะเป็นศูนย์

อัลกอริทึม 7 (ความลึกของจุดยอดในต้นไม้)
ข้อมูลเข้า: ต้นไม้มีราก T ที่มี n จุดยอด ในรูปแทนทัวร์ออยเลอร์ด้วยอาร์เรย์สองตัว $L[1..n]$ และ $R[1..n]$
ผลลัพธ์: อาร์เรย์ $D[1..n]$ ที่เก็บความลึกของจุดยอดแต่ละจุดของ T

```
1  integer A[1..2n]
2  forall i ∈ 1:n do
3     A[L[i]] ← 1
4     A[R[i]] ← −1
5  enddo
6  B ← Excl-Scan(A, +)
7  forall i ∈ 1:n do
8     D[i] ← B[L[i]]
9  enddo
```

อัลกอริทึม 7 แสดงวิธีใช้สมบัตินี้หาความลึกของจุดยอดแต่ละจุดในต้นไม้ ในอัลกอริทึมนี้ ต้นไม้ T ที่มี n จุดยอดถูกแทนด้วยอาร์เรย์สองตัว L และ R ยาว n ในหน่วยความจำร่วม (สำหรับวงเล็บเปิดและวงเล็บปิดตามลำดับ) อาร์เรย์เหล่านี้ต้องสร้างจากทัวร์ออยเลอร์ที่เริ่มต้นและสิ้นสุดที่ราก L_i คือตำแหน่งแรกสุดในทัวร์ที่ไปเยือนจุดยอด i และ R_i คือตำแหน่งสุดท้ายในทัวร์ที่ไปเยือนจุดยอด i

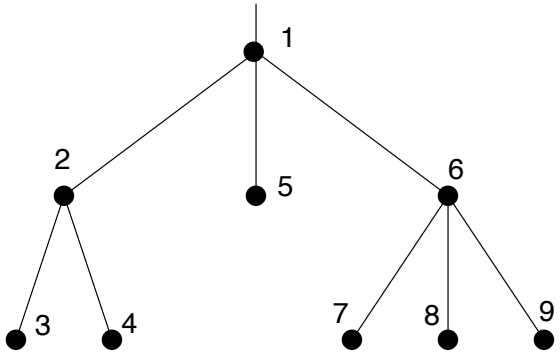
รูปที่ 3 แสดงการทำงานของอัลกอริทึม 7 อัลกอริทึมนี้ทำงานบน EREW PRAM ด้วยความซับซ้อนเชิงขั้น $\Theta(\lg n)$ และความซับซ้อนเชิงงาน $\Theta(n)$

ต่อไปเราจะอธิบายวิธีสร้างทัวร์ออยเลอร์จากการแทนต้นไม้ T ด้วยตัวชี้ เราสมมติว่าเส้นเชื่อม $2n - 2$ เส้นของ T' ถูกแทนเป็นเซตของลิสต์ประชิด (adjacency list) ของแต่ละจุดยอด และสมมติต่อไปว่าลิสต์ประชิด L_u ของจุดยอด u เชื่อมโยงกันเป็นวง (circularly linked) ดังแสดงในรูปที่ 4 สมาชิก v ของลิสต์ L_u นิยามเส้นเชื่อม (u, v) เส้นเชื่อมสมมาตร (v, u) อยู่ในลิสต์ L_v เราสมมติว่าเส้นเชื่อมสมมาตรเชื่อมถึงกันด้วยตัวชี้ทั้งสองทิศทาง (แสดงด้วยลูกศรเส้นประในรูปที่ 4) ดังนั้นในลิสต์ประชิดจึงมีเส้นเชื่อมทั้งหมด $2n-2$ เส้น และเราสมมติว่าสมาชิกเหล่านี้จัดเก็บอยู่ในอาร์เรย์ E' ขนาด $2n - 2$

ถ้าเราพิจารณาเพื่อนบ้านของจุดยอด u ตามลำดับที่ปรากฏใน $L_u = (v_0, \ldots, v_{d-1})$ เมื่อ d คือดีกรีของจุดยอด u และนิยามฟังก์ชันตัวตามหลังดังนี้: $s((v_i, u)) = (u, v_{(i+1) \bmod d})$ เราจะได้ทัวร์ออยเลอร์ที่ถูกต้องของ T (เพื่อพิสูจน์ เราต้องแสดงว่าเราสร้างวัฏจักรเดียว ไม่ใช่เซตของวัฏจักรที่ไม่มีส่วนร่วมกัน ข้อเท็จจริงนี้พิสูจน์ได้ด้วยการอุปนัยบนจำนวนจุดยอด)

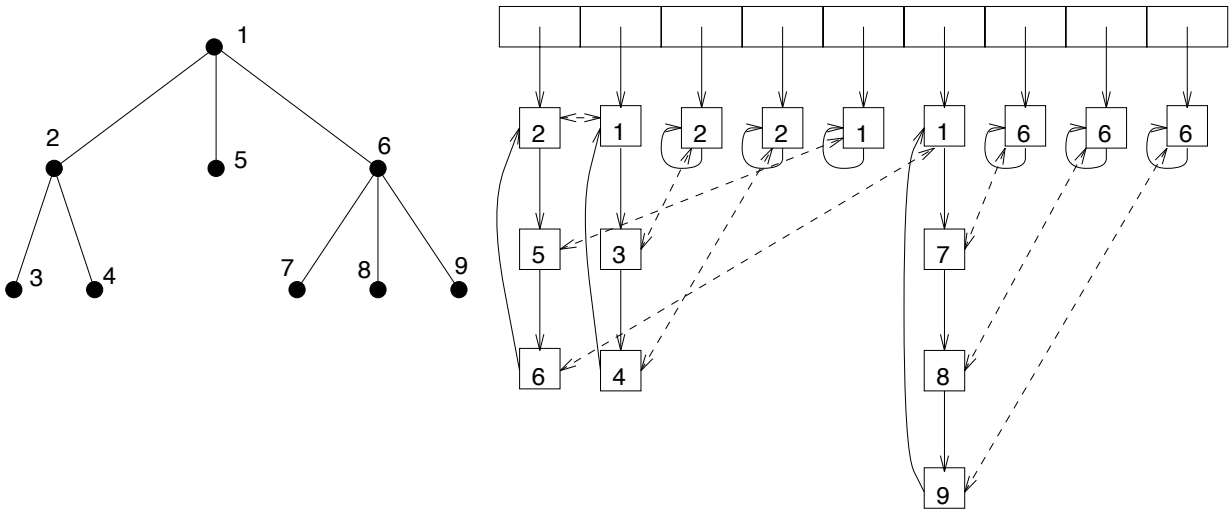
ฟังก์ชันตัวตามหลังประเมินค่าแบบขนานสำหรับเส้นเชื่อมแต่ละเส้นใน E' ได้ โดยตามตัวชี้สมมาตร (เส้นประ) แล้วตามด้วยตัวชี้ของลิสต์ประชิด (เส้นทึบ) ซึ่งใช้ $\Theta(1)$ ขั้นด้วยความซับซ้อนเชิงงาน $\Theta(n)$ จึงมีประสิทธิภาพเชิงงาน นอกจากนี้ เนื่องจากตัวชี้สองตัวที่ตามสำหรับสมาชิกแต่ละตัวใน E' ไม่ซ้ำกัน เราจึงทำสิ่งนี้บน EREW PRAM ได้

สังเกตว่าสิ่งที่เราทำสำเร็จคือการเชื่อมเส้นเชื่อมของ T' ให้เป็นทัวร์ออยเลอร์ หากต้องการสร้างรูปแทนอย่างอาร์เรย์ L และ R ที่ใช้ในอัลกอริทึม 7 เราต้องทำงานเพิ่มเติมอีก



$$
\begin{array}{lcccccccccccccccccc}
L = & 1 & 2 & 3 & 5 & 8 & 10 & 11 & 13 & 15 & & & & & & & & & \\
R = & 18 & 7 & 4 & 6 & 9 & 17 & 12 & 14 & 16 & & & & & & & & & \\
A = & 1 & 1 & 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 & 1 & -1 & 1 & -1 & -1 & -1 \\
B = & 0 & 1 & 2 & 3 & 2 & 3 & 2 & 1 & 2 & 1 & 2 & 3 & 2 & 3 & 2 & 3 & 2 & 1 \\
D = & 0 & 1 & 2 & 2 & 1 & 1 & 2 & 2 & 2 & & & & & & & & &
\end{array}
$$

รูปที่ 3: การหาความลึกของจุดยอดในต้นไม้



รูปที่ 4: การสร้างรูปแทนทัวร์ออยเลอร์ของต้นไม้จากรูปแทนด้วยตัวชี้

Claude อธิบายเพิ่ม: อ่านรูปที่ 3 อย่างไร

ทัวร์ออยเลอร์ของต้นไม้ในรูปคือการเดินรอบต้นไม้แบบ “เดินเลียบกำแพง” เขียนวงเล็บเปิดเมื่อเข้าจุดยอดครั้งแรก และวงเล็บปิดเมื่อออกครั้งสุดท้าย:

$$
\begin{array}{c}
(\; (\; (\;) \; (\;) \;) \; (\;) \; (\; (\;) \; (\;) \; (\;) \;) \;) \\
1 \; 2 \; 3 \; 3 \; 4 \; 4 \; 2 \; 5 \; 5 \; 6 \; 7 \; 7 \; 8 \; 8 \; 9 \; 9 \; 6 \; 1
\end{array}
$$

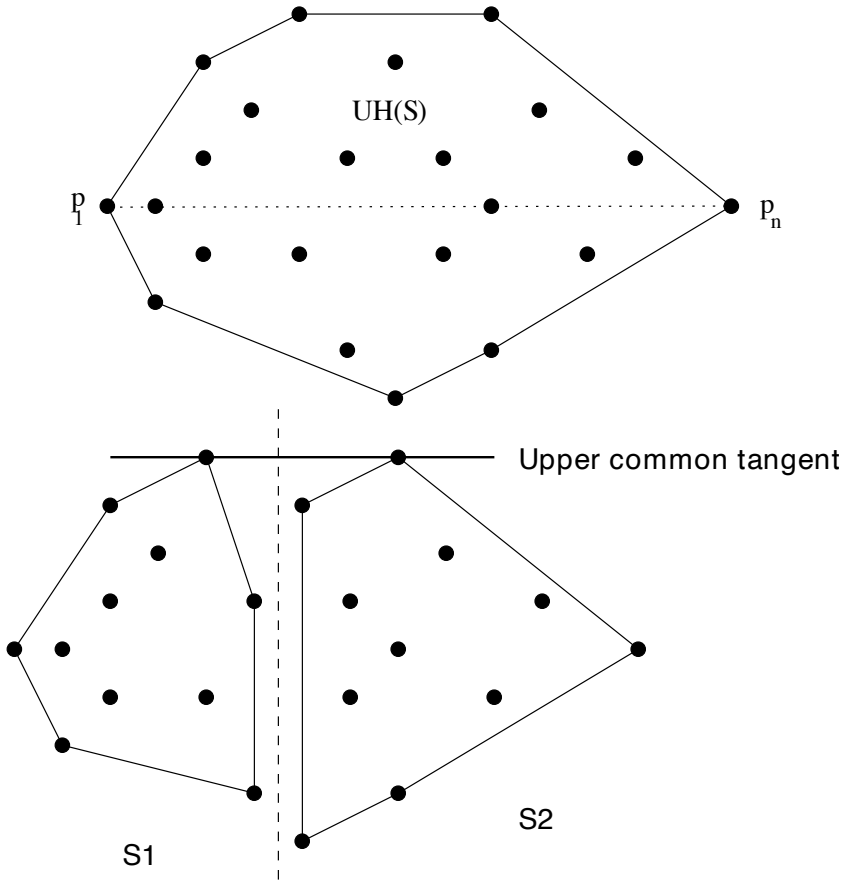
แถว A คือ $+1$ ที่วงเล็บเปิดและ -1 ที่วงเล็บปิด ผลบวกสะสมแบบ exclusive (B) ที่ตำแหน่งวงเล็บเปิดของจุดยอดใดก็คือ “จำนวนวงเล็บที่ยังเปิดค้างอยู่” ซึ่งก็คือความลึกของจุดยอดนั้นพอดี ตัวอย่างเช่น จุดยอด 7 เปิดที่ตำแหน่ง $L_7 = 11$ ได้ $D_7 = B_{11} = 2$

3.5 การแบ่งแยกและเอาชนะ

กลยุทธ์การแบ่งแยกและเอาชนะ (divide-and-conquer) เป็นกลยุทธ์ที่คุ้นเคยจากการคำนวณแบบลำดับ ประกอบด้วยสามขั้นตอน คือ แบ่งปัญหาเป็นปัญหาย่อย แก้ปัญหาย่อยแบบเวียนเกิด และรวมผลเฉลยย่อยให้เป็นผลเฉลย เช่นเคย ขั้นตอนที่หนึ่งและสามเป็นขั้นตอนสำคัญ

เพื่อแสดงกลยุทธ์นี้ในบริบทแบบขนาน เราพิจารณาปัญหาเปลือกนูนบนระนาบ (planar convex hull) กำหนดเซต $S = \{p_1, \ldots, p_n\}$ ของจุด โดยแต่ละจุด p_i เป็นคู่อันดับของพิกัด (x_i, y_i) เราสมมติเพิ่มว่าจุดถูกเรียงตามพิกัด x แล้ว (ถ้าไม่ ก็ทำเป็นขั้นตอนเตรียมการได้ด้วยขอบเขตความซับซ้อนที่ต่ำพอ) เราต้องการหาเปลือกนูน $\mathrm{CH}(S)$ ซึ่งคือรูปหลายเหลี่ยมนูนที่เล็กที่สุดที่บรรจุจุดทั้งหมดของ S โดยไล่จุดยอดของรูปหลายเหลี่ยมนี้ตามเข็มนาฬิกา รูปที่ 5 แสดงตัวอย่างหนึ่งของปัญหานี้

ความซับซ้อนแบบลำดับของปัญหานี้คือ $T_S(n) = \Theta(n \lg n)$ อัลกอริทึมที่เป็นที่รู้จักดีหลายตัวสำหรับปัญหานี้ให้ขอบเขตบน และการลดรูป (reduction) จากการเรียงลำดับแบบเปรียบเทียบให้ขอบเขตล่าง ดูรายละเอียดใน §35.3 ของ CLR



รูปที่ 5: การหาเปลือกนูนของเซตของจุด (Upper common tangent = เส้นสัมผัสร่วมบน)

ข้อแรก เราสังเกตว่า p_1 และ p_n เป็นส่วนหนึ่งของ $\mathrm{CH}(S)$ เพราะ S ถูกเรียงไว้แล้ว และสองจุดนี้แบ่งเปลือกนูนออกเป็น*เปลือกบน* (upper hull) $\mathrm{UH}(S)$ และ*เปลือกล่าง* (lower hull) $\mathrm{LH}(S)$ โดยไม่เสียนัยทั่วไป เราจะแสดงวิธีคำนวณ $\mathrm{UH}(S)$

ขั้นตอนการแบ่งทำได้ง่าย: เราแบ่ง S เป็น $S_1 = \{p_1, \ldots, p_{n/2}\}$ และ $S_2 = \{p_{n/2+1}, \ldots, p_n\}$ จากนั้นหา $\mathrm{UH}(S_1) = \langle p_1 = q_1, \ldots, q_s \rangle$ และ $\mathrm{UH}(S_2) = \langle r_1, \ldots, r_t = p_n \rangle$ แบบเวียนเกิด สมมติว่าเมื่อ $n \leqslant 4$ เราแก้ปัญหาด้วยวิธีถึก (brute force) ซึ่งให้เงื่อนไขหยุดของการเวียนเกิด

ขั้นตอนการรวมไม่ง่ายนัก ให้*เส้นสัมผัสร่วมบน* (upper common tangent: UCT) เป็นเส้นสัมผัสร่วมของ $\mathrm{UH}(S_1)$ และ $\mathrm{UH}(S_2)$ ที่ทำให้ทั้ง $\mathrm{UH}(S_1)$ และ $\mathrm{UH}(S_2)$ อยู่ใต้เส้นนั้น ดังนั้นเส้นสัมผัสนี้ประกอบด้วยสองจุด จุดหนึ่งจาก $\mathrm{UH}(S_1)$ และอีกจุดจาก $\mathrm{UH}(S_2)$ ให้ $\mathrm{UCT}(S_1, S_2) = (q_i, r_j)$ สมมติว่ามีอัลกอริทึมแบบลำดับที่ใช้เวลา $O(\lg n)$ สำหรับหา $\mathrm{UCT}(S_1, S_2)$ (ดู Preparata and

Shamos, Lemma 3.1) แล้ว $\mathrm{UH}(S) = \langle q_1, \ldots, q_i, r_j, \ldots, r_t \rangle$ และมี $(i + t - j + 1)$ จุด เมื่อรู้ s, t, i และ j เราได้ $\mathrm{UH}(S)$ ใน $\Theta(1)$ ขั้นและงาน $O(n)$ ดังนี้

forall $k \in 1 : i + t - j + 1$ do
  $\mathrm{UH}(S)_k \leftarrow$ if $k \leqslant i$ then q_k else $r_{k+j-i-1}$ endif
enddo

อัลกอริทึมนี้ต้องการแบบจำลองขั้นต่ำเป็น CREW PRAM ในการวิเคราะห์ความซับซ้อน เราสังเกตว่า

$$S(n) = S(n/2) + O(\lg n) \tag{9}$$
$$W(n) = 2W(n/2) + O(n) \tag{10}$$

ซึ่งให้ $S(n) = O(\lg^2 n)$ และ $W(n) = O(n \lg n)$

Claude อธิบายเพิ่ม: ทำไมต้องเป็น CREW

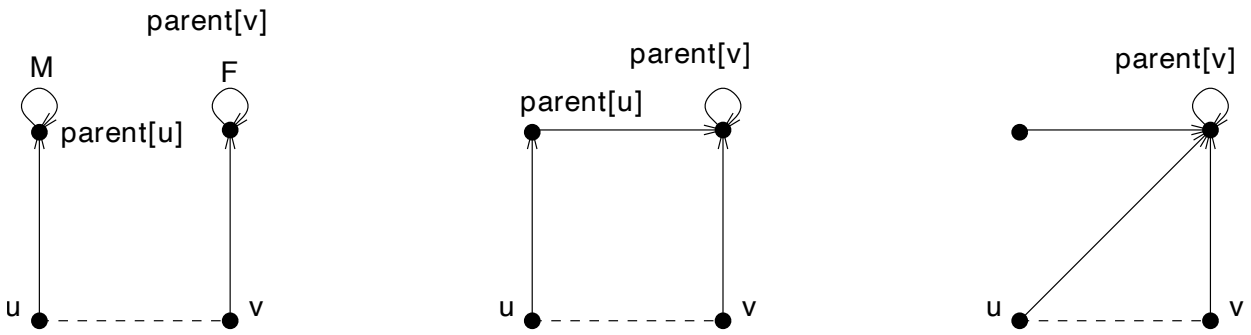
เส้นสัมผัสร่วมบนหาได้ด้วยโพรเซสเซอร์ตัวเดียว แต่หลังจากนั้นโพรเซสเซอร์ทุกตัวในขั้นรวมต้อง*อ่าน* i และ j ตัวเดียวกันพร้อมกัน เพื่อรู้ว่าตัวเองต้องลอกจุดจากฝั่งไหน การอ่านพร้อมกันนี้ทำให้ต้องใช้ CREW (บน EREW จะต้องกระจายค่า i, j ก่อน ซึ่งใช้เวลา $O(\lg n)$ และไม่ทำให้ $S(n)$ แย่ลงเชิงเส้นกำกับ)

3.6 การทำลายความสมมาตร

เทคนิคการทำลายความสมมาตร (symmetry breaking) ใช้ในอัลกอริทึมบน PRAM เพื่อแยกความแตกต่างระหว่างสมาชิกที่ดูเหมือนกัน ซึ่งทำได้ทั้งแบบเชิงกำหนดและแบบเชิงความน่าจะเป็น เราจะศึกษาอัลกอริทึมแบบสุ่ม (ที่รู้จักกันในชื่อ*อัลกอริทึม random mate*) สำหรับหาส่วนประกอบเชื่อมโยง (connected components) ของกราฟไม่มีทิศทาง เพื่อเป็นตัวอย่างของเทคนิคนี้ ดู §30.5 ของ CLR สำหรับตัวอย่างอัลกอริทึมการทำลายความสมมาตรแบบเชิงกำหนด

ให้ $G = (V, E)$ เป็นกราฟไม่มีทิศทาง เรากล่าวว่าเส้นเชื่อม (u, v) *ตกกระทบ* (hits) จุดยอด u และ v *ดีกรี*ของจุดยอด v คือจำนวนเส้นเชื่อมที่ตกกระทบ v *เส้นทาง*จาก v_1 ไป v_k (เขียนแทนด้วย $v_1 \rightsquigarrow v_k$) คือลำดับของจุดยอด $(v_1, \ldots, v_k)$ ซึ่ง $(v_i, v_{i+1}) \in E$ สำหรับ $1 \leqslant i < k$ *กราฟย่อยเชื่อมโยง* (connected subgraph) คือเซตย่อย U ของ V ซึ่งสำหรับทุก $u, v \in U$ เรามี $u \rightsquigarrow v$ *ส่วนประกอบเชื่อมโยง* คือกราฟย่อยเชื่อมโยงที่ใหญ่ที่สุด (maximal) *ซูเปอร์เวอร์เท็กซ์* (supervertex) คือโครงสร้างข้อมูลต้นไม้มีทิศทางแบบมีรากที่ใช้แทนกราฟย่อยเชื่อมโยง เราใช้ข้อตกลงมาตรฐานของเซตไม่มีส่วนร่วม คือเส้นเชื่อมชี้จากลูกไปยังพ่อแม่ และใช้วงวนตนเองที่รากเพื่อแทนซูเปอร์เวอร์เท็กซ์

ในแบบจำลองลำดับ เราหาส่วนประกอบเชื่อมโยงได้อย่างเหมาะที่สุดด้วยการค้นหาแนวลึก ดังนั้น $T_S(V, E) = O(V + E)$ อัลกอริทึมแบบขนานของเราจะคล้ายกับอัลกอริทึมใน §22.1 ของ CLR แนวคิดเบื้องหลังอัลกอริทึมคือการรวมซูเปอร์เวอร์เท็กซ์เข้าด้วยกันให้ได้ซูเปอร์เวอร์เท็กซ์ที่ใหญ่ขึ้น ในกรณีแบบลำดับ เราตรวจสอบเส้นเชื่อมตามลำดับที่กำหนดไว้ล่วงหน้า สำหรับอัลกอริทึมแบบขนาน เราต้องการตรวจสอบเส้นเชื่อมหลายเส้นในแต่ละขั้นเวลา เราทำลายความสมมาตรโดยเลือกซูเปอร์เวอร์เท็กซ์ถัดไปที่จะรวมแบบพลการ ด้วยการสุ่มกำหนดเพศ (gender) ให้กับซูเปอร์เวอร์เท็กซ์ เราเรียกเส้นเชื่อม (u, v) ของกราฟว่า*ยังมีชีวิต* (live) ถ้า u และ v อยู่ในซูเปอร์เวอร์เท็กซ์ต่างกัน และเรียกซูเปอร์เวอร์เท็กซ์ว่ายังมีชีวิต ถ้ามีเส้นเชื่อมที่ยังมีชีวิตอย่างน้อยหนึ่งเส้นตกกระทบจุดยอดบางจุดของซูเปอร์เวอร์เท็กซ์นั้น ตราบใดที่ยังมีเส้นเชื่อมที่ยังมีชีวิต เราจะรวมซูเปอร์เวอร์เท็กซ์ที่มีเพศตรงข้ามกันและเชื่อมกันด้วยเส้นเชื่อมที่ยังมีชีวิต การรวมนี้มีขั้นตอนการบีบอัดเส้นทาง (path compression) อยู่ด้วย เมื่อไม่มีเส้นเชื่อมที่ยังมีชีวิตเหลือแล้ว เราก็ได้ส่วนประกอบเชื่อมโยง



รูปที่ 6: รายละเอียดของขั้นตอนการรวมในอัลกอริทึม 8 เส้นเชื่อมของกราฟไม่มีทิศทางและแสดงด้วยเส้นประ เส้นเชื่อมของซูเปอร์เวอร์เท็กซ์มีทิศทางและแสดงด้วยเส้นทึบ (M = เพศชาย, F = เพศหญิง)

อัลกอริทึม 8 (อัลกอริทึม random mate สำหรับส่วนประกอบเชื่อมโยง)
ข้อมูลเข้ากราฟไม่มีทิศทาง $G = (V, E)$
ผลลัพธ์: ส่วนประกอบเชื่อมโยงของ G โดยระบุหมายเลขไว้ในอาร์เรย์ $P[1..|V|]$

```
1   forall v ∈ V do
2     parent[v] ← v
3   enddo
4   while มีเส้นเชื่อมที่ยังมีชีวิตใน G do
5     forall v ∈ V do
6       gender[v] ← rand({M, F})   [แก้ 2]
7     enddo
8     forall (u, v) ∈ E | live(u, v) do
9       if gender[parent[u]] = M and gender[parent[v]] = F then
10        parent[parent[u]] ← parent[v]
11      endif
12      if gender[parent[v]] = M and gender[parent[u]] = F then
13        parent[parent[v]] ← parent[u]
14      endif
15    enddo
16    forall v ∈ V do
17      parent[v] ← parent[parent[v]]
18    enddo
19  endwhile
```

รูปที่ 6 แสดงรายละเอียดของขั้นตอนการรวมในอัลกอริทึม 8 เราจะหาความซับซ้อนของอัลกอริทึมนี้โดยพิสูจน์บทตั้งชุดหนึ่งเกี่ยวกับพฤติกรรมของมัน

บทตั้ง 1. *หลังการวนแต่ละรอบของลูป* while *ชั้นนอก ซูเปอร์เวอร์เท็กซ์แต่ละตัวเป็นดาว (star) (ต้นไม้ที่สูงศูนย์หรือหนึ่ง)*

พิสูจน์. พิสูจน์โดยอุปนัยบนจำนวนรอบที่วนไปแล้ว ก่อนที่ลูปจะวนรอบใดเลย จุดยอดแต่ละจุดเป็นซูเปอร์เวอร์เท็กซ์ที่มีความสูงศูนย์จากการกำหนดค่าเริ่มต้นในบรรทัดที่ 2 ต่อไปสมมติว่าข้อความนี้เป็นจริงหลังการวน k รอบ แล้วพิจารณาสิ่งที่เกิดขึ้นในรอบที่ $(k + 1)$ ดูรูปที่ 6 หลังลูป forall ในบรรทัดที่ 8 ความสูงของซูเปอร์เวอร์เท็กซ์อาจเพิ่มขึ้นหนึ่ง จึงสูงไม่เกินสอง หลังขั้นตอนการบีบอัดในบรรทัดที่ 16 ความสูงจะลดจากสองกลับมาเป็นหนึ่ง □

บทตั้ง 2. *การวนแต่ละรอบของลูป* while *ใช้* $\Theta(1)$ *ขั้นและงาน* $O(V + E)$

พิสูจน์. ข้อนี้ง่าย ส่วนเดียวที่ไม่ชัดเจนคือการหาเส้นเชื่อมที่ยังมีชีวิต ซึ่งทำได้ใน $\Theta(1)$ ขั้นและงาน $O(E)$ เนื่องจากจุดยอดแต่ละจุดเป็นดาวตามบทตั้งที่ 1 เส้นเชื่อม (u, v) จะยังมีชีวิตก็ต่อเมื่อ $\text{parent}[u] \neq \text{parent}[v]$ □

บทตั้ง 3. *ความน่าจะเป็นที่ในรอบหนึ่ง ซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตจะถูกรวมเข้ากับซูเปอร์เวอร์เท็กซ์อื่นคือ* $\geqslant 1/4$

พิสูจน์. ซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตมีเส้นเชื่อมที่ยังมีชีวิตอย่างน้อยหนึ่งเส้น ซูเปอร์เวอร์เท็กซ์นั้นจะได้รากใหม่ก็ต่อเมื่อเพศของมันคือ M และมีเส้นเชื่อมที่ยังมีชีวิตไปยังซูเปอร์เวอร์เท็กซ์ที่มีเพศ F ความน่าจะเป็นของเหตุการณ์นี้คือ $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$ ความน่าจะเป็นจริงมีค่าอย่างน้อยเท่านี้ เพราะซูเปอร์เวอร์เท็กซ์อาจมีเส้นเชื่อมที่ยังมีชีวิตมากกว่าหนึ่งเส้น □

บทตั้ง 4. *ความน่าจะเป็นที่จุดยอดหนึ่งจะเป็นรากที่ยังมีชีวิตหลังการวนลูป* while *ไป* $5 \lg |V|$ *รอบคือ* $\leqslant 1/|V|^2$

พิสูจน์. จากบทตั้งที่ 3 ซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตในรอบที่ i มีความน่าจะเป็น $\leqslant \frac{3}{4}$ ที่จะยังมีชีวิตอยู่หลังรอบที่ $i + 1$ ดังนั้นความน่าจะเป็นที่มันยังมีชีวิตหลังการวน $5 \lg |V|$ รอบคือ $\leqslant (\frac{3}{4})^{5 \lg |V|}$ อสมการตามมาเพราะ $\lg(\frac{3}{4})^{5 \lg |V|} = -2.075 \lg |V|$ และ $\lg \frac{1}{|V|^2} = -2 \lg |V|$ □

บทตั้ง 5. *ค่าคาดหมายของจำนวนซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตหลังการวน* $5 \lg |V|$ *รอบคือ* $\leqslant 1/|V|$

พิสูจน์. เราคำนวณค่าคาดหมายของจำนวนซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตโดยรวมความน่าจะเป็นที่จุดยอดแต่ละจุดจะเป็นรากที่ยังมีชีวิต จากบทตั้งที่ 4 ค่านี้คือ $\leqslant |V| \cdot \frac{1}{|V|^2} = \frac{1}{|V|}$ □

ทฤษฎีบท 3. *ด้วยความน่าจะเป็นไม่เกิน* $1/|V|$ *อัลกอริทึมจะยังไม่สิ้นสุดหลังการวน* $5 \lg |V|$ *รอบ*

พิสูจน์. ให้ p_k เป็นความน่าจะเป็นที่จะมีซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิต k ตัวหลังการวน $5 \lg |V|$ รอบ ตามนิยามของค่าคาดหมาย ค่าคาดหมายของจำนวนซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตหลังการวน $5 \lg |V|$ รอบคือ $\sum_{k=0}^{|V|} k \cdot p_k$ และจากบทตั้งที่ 5 ค่านี้คือ $\leqslant \frac{1}{|V|}$ เนื่องจาก k และ p_k เป็นบวกทั้งหมด จึงได้ $\sum_{k=1}^{|V|} p_k \leqslant \sum_{k=0}^{|V|} k \cdot p_k \leqslant \frac{1}{|V|}$ อัลกอริทึมจะสิ้นสุดเมื่อจำนวนซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตเป็นศูนย์ ดังนั้น $\sum_{k=1}^{|V|} p_k$ คือความน่าจะเป็นที่ยังต้องทำงานต่อหลังการวน $5 \lg |V|$ รอบ[แก้ 3] □

อัลกอริทึม random mate ต้องการแบบจำลอง CRCW PRAM การเขียนพร้อมกันเกิดขึ้นในขั้นตอนการรวม (บรรทัดที่ 8) เพราะจุดยอดต่างกันอาจมีพ่อแม่ร่วมกัน

ความซับซ้อนเชิงขั้นของอัลกอริทึมคือ $O(\lg V)$ ด้วยความน่าจะเป็นสูง ซึ่งเป็นผลจากทฤษฎีบทที่ 3 ความซับซ้อนเชิงงานคือ $O((V + E) \lg V)$ จากทฤษฎีบทที่ 3 และบทตั้งที่ 2 ดังนั้นอัลกอริทึม random mate จึงไม่เหมาะที่สุดเชิงงาน (work-optimal)

ปัจจัยสำคัญในอัลกอริทึมนี้คือเส้นทางในซูเปอร์เวอร์เท็กซ์สั้น (อันที่จริงคือ $\Theta(1)$) ทำให้ซูเปอร์เวอร์เท็กซ์หลังการรวมกลับเป็นดาวได้ด้วยการบีบอัดเส้นทางเพียงรอบเดียวในบรรทัดที่ 16 ถ้าเราใช้อัลกอริทึมแบบเชิงกำหนดบางตัวเพื่อทำลายความสมมาตร เราจะรับประกันไม่ได้ว่าเส้นทางจะสั้น เราอาจได้ซูเปอร์เวอร์เท็กซ์หลายตัวและเส้นทางยาวภายในซูเปอร์เวอร์เท็กซ์ และความซับซ้อนเชิงขั้นของอัลกอริทึมเช่นนั้นจะเป็น $O(\lg^2 V)$ มีอัลกอริทึมแบบเชิงกำหนดของ Shiloach และ Vishkin ที่หลีกเลี่ยงปัญหานี้โดยไม่บีบอัดเส้นทางทั้งหมดในแต่ละขั้น แต่รักษาชุดค่าคงตัว (invariants) ที่ซับซ้อนซึ่งรับประกันว่าซูเปอร์เวอร์เท็กซ์ที่เหลืออยู่เมื่ออัลกอริทึมสิ้นสุดแทนส่วนประกอบเชื่อมโยงได้ถูกต้องจริง

Claude อธิบายเพิ่ม: ทำไมต้องสุ่มเพศ

ถ้าทุกซูเปอร์เวอร์เท็กซ์ “ชี้ไปหาเพื่อนบ้าน” พร้อมกันโดยไม่มีกติกา อาจเกิดสายโซ่ยาว $a \to b \to c \to \cdots$ หรือแม้แต่วัฏจักร $a \to b \to a$ ได้ การให้*เฉพาะ M ไปเกาะ F* ทำให้ F ไม่ขยับในรอบนั้นเลย ผลลัพธ์จึงเป็นดาวสูงไม่เกินสองชั้นเสมอ (บทตั้งที่ 1) ราคาที่ต้องจ่ายคือโอกาส “คู่ไม่ลงตัว” (M–M หรือ F–F) แต่บทตั้งที่ 3 รับประกันว่าแต่ละรอบมีโอกาสอย่างน้อย $1/4$ ที่จะรวมได้ จำนวนซูเปอร์เวอร์เท็กซ์ที่ยังมีชีวิตจึงลดลงแบบเรขาคณิต และหมดภายใน $O(\lg V)$ รอบ

4 ทัวร์อัลกอริทึมแบบขนานเชิงข้อมูล

ในหัวข้อนี้ เราจะนำเสนออัลกอริทึมแบบขนานเชิงข้อมูล (data-parallel algorithms) หลากหลายตัวสำหรับปัญหาที่พบบ่อยบางปัญหา

4.1 อัลกอริทึมพื้นฐานที่ใช้ scan

เราสร้างส่วนประกอบพื้นฐาน (building blocks) ที่มีประโยชน์ได้หลายตัวโดยใช้การดำเนินการ scan เป็นตัวดำเนินการพื้นฐาน (primitive) ในตัวอย่างต่อไปนี้ เราจะใช้ทั้งการระบุดัชนีอาร์เรย์แบบเริ่มที่ศูนย์และแบบเริ่มที่หนึ่ง ในแต่ละกรณี อย่าลืมคำนวณความซับซ้อนเชิงขั้นและเชิงงาน และแบบจำลอง PRAM ขั้นต่ำที่ต้องใช้

Enumerate การดำเนินการ *enumerate* รับเวกเตอร์บูลีน แล้วกำหนดหมายเลขให้สมาชิกที่เป็น true ดังนี้

```
sequence⟨integer⟩ ENUMERATE(sequence⟨boolean⟩ Flag)
1   forall i ∈ 1 : #Flag do
2     V[i] ← if Flag[i] then 1 else 0 endif
3   enddo
4   return SCAN(V, +)
```

Flag =	true	true	false	true	false	false	true
V =	1	1	0	1	0	0	1
ผลลัพธ์ =	1	2	2	3	3	3	4

Copy การดำเนินการ *copy* (หรือ *distribute*) คัดลอกค่าจำนวนเต็มหนึ่งค่าไปทั่วทั้งอาร์เรย์

```
sequence⟨integer⟩ COPY(integer v, integer n)
1   forall i ∈ 1 : n do
2     V[i] ← if i = 1 then v else 0 endif
3   enddo
4   return SCAN(V, +)
```

v =	5						
n =	7						
V =	5	0	0	0	0	0	0
ผลลัพธ์ =	5	5	5	5	5	5	5

Pack การดำเนินการ *pack* รับเวกเตอร์ค่า A และเวกเตอร์บูลีนของแฟล็ก F แล้วคืนเวกเตอร์ที่มีเฉพาะสมาชิกของ A ที่แฟล็กตรงกันเป็น true

```
sequence⟨T⟩ PACK(sequence⟨T⟩ A, sequence⟨boolean⟩ F)
1   P ← ENUMERATE(F)
2   forall i ∈ 1 : #F do
3     if F[i] then
4       R[P[i]] ← A[i]
5     endif
6   enddo
7   return R[1 : P[#F]]
```

$A =$	♣	♢	♡	♠	▲	■	♦
$F =$	true	true	false	true	false	false	true
$P =$	1	2	2	3	3	3	4
$R =$	♣	♢	♠	♦			

Split การดำเนินการ *split* รับเวกเตอร์ค่า A และเวกเตอร์บูลีนของแฟล็ก F แล้วคืนเวกเตอร์ที่ย้ายสมาชิกที่มีแฟล็ก false ไปไว้ด้านล่าง และย้ายสมาชิกที่มีแฟล็ก true ไปไว้ด้านบน

```
sequence⟨T⟩ SPLIT(sequence⟨T⟩ A, sequence⟨boolean⟩ F)
1   Down ← ENUMERATE(not (F))
2   P ← ENUMERATE(F)
3   forall i ∈ 1 : #F do
4     Index[i] ← if F[i] then P[i] + Down[#F] else Down[i] endif
5   enddo
6   forall i ∈ 1 : #F do
7     R[Index[i]] ← A[i]
8   enddo
9   return R
```

$A =$	♣	♢	♡	♠	▲	■	♦
$F =$	true	true	false	true	false	false	true
Down $=$	0	0	1	1	2	3	3
Index $=$	4	5	1	6	2	3	7
$R =$	♡	▲	■	♣	♢	♠	♦

การ split แบบขนานนี้ใช้เป็นแกนหลักของการเรียงลำดับแบบฐาน (radix sort) แบบขนานได้ สังเกตว่า split มีความเสถียร (stable) ซึ่งเป็นเรื่องสำคัญยิ่ง

Claude อธิบายเพิ่ม: ส่วนประกอบทั้งสี่เป็นสูตรเดียวกัน

ทุกตัวมีโครงเดียวกันคือ "**map** ทีละตัว (งาน $O(n)$, $O(1)$ ขั้น) → **scan** (งาน $O(n)$, $O(\lg n)$ ขั้น) → **map/เขียนตามดัชนี**" จึงได้งาน $O(n)$ และ $O(\lg n)$ ขั้นทั้งหมด หัวใจอยู่ที่ scan เปลี่ยน "ฉันถูกเลือกไหม" (0/1) ให้เป็น "ฉันเป็นตัวที่ถูกเลือกลำดับที่เท่าไร" ซึ่งก็คือตำแหน่งปลายทางที่ไม่ชนกัน การเขียนจึงเป็นแบบเอกสิทธิ์ (ส่วน Copy คือ scan ที่ทุกตัวได้ค่า v เท่ากัน จึงกระจายค่าได้โดยไม่ต้องอ่านพร้อมกัน)

ความเสถียรของ split สำคัญกับ radix sort เพราะ radix sort เรียงทีละบิตจากบิตต่ำไปบิตสูง ถ้า split สลับลำดับของสมาชิกที่บิตปัจจุบันเท่ากัน ผลการเรียงจากบิตก่อนหน้าจะเสียไป

4.2 การวิเคราะห์ศัพท์แบบขนาน

ตัวอย่างสุดท้ายของเราคือตัววิเคราะห์ศัพท์แบบขนาน (parallel lexical analyzer) ซึ่งเราสร้างโดยใช้ scan ปัญหาคือการแบ่งสายอักขระบนตัวอักษรชุดหนึ่ง (alphabet) ออกเป็นโทเค็น (tokens) ตามโครงสร้างศัพท์ที่ระบุด้วยภาษาปรกติ (regular language) และรู้จำโดยออโตมาตาจำกัดเชิงกำหนด (deterministic finite automaton: DFA) จำได้ว่าในเชิงรูปนัย DFA M คือ 5-สิ่งอันดับ (5-tuple) $M = (S, \Sigma, \delta, q_0, F)$ เมื่อ S เป็นเซตของสถานะ Σ เป็นตัวอักษรของข้อมูลเข้า $\delta : S \times \Sigma \to S$ เป็นฟังก์ชันการเปลี่ยนสถานะ $q_0 \in S$ เป็นสถานะเริ่มต้น และ $F \subseteq S$ เป็นเซตของสถานะสุดท้าย การใช้ scan ในตัวอย่างนี้น่าสนใจตรงที่ตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่ที่ใช้ (การประกอบฟังก์ชัน: function composition) ไม่มีสมบัติการสลับที่ (noncommutative)

พิจารณาวงศ์ของฟังก์ชัน $\{f_i : S \to S \mid i \in \Sigma\}$ โดยที่ $f_i(s) = \delta(s, i)$ นั่นคือ ฟังก์ชัน f_i อธิบายการทำงานของ DFA M เมื่อได้รับสัญลักษณ์ข้อมูลเข้า i หรือกล่าวอีกแบบหนึ่ง สัญลักษณ์ข้อมูลเข้าแต่ละตัวถูกเข้ารหัสเป็นฟังก์ชัน สังเกตว่าวงศ์ของฟังก์ชัน $\{f_i\}$

เป็นการแบ่งกั้น (partition) ของ δ อันที่จริง ฟังก์ชันเหล่านี้ก็คือคอลัมน์ของตารางการเปลี่ยนสถานะของ DFA นั่นเอง เราแทน f_i แต่ละตัวได้ด้วยอาร์เรย์หนึ่งมิติของสถานะที่ใช้สถานะเป็นดัชนี เนื่องจากการประกอบฟังก์ชันเป็นตัวดำเนินการทวิภาคที่มีสมบัติการเปลี่ยนหมู่ เราจึงนิยาม scan บนตัวดำเนินการนี้ได้ และนี่คือกุญแจของอัลกอริทึมนี้ การประกอบฟังก์ชันที่แทนด้วยอาร์เรย์ทำได้โดยแทนที่สมาชิกแต่ละตัวของอาร์เรย์หนึ่งด้วยผลของการใช้สมาชิกนั้นเป็นดัชนีเข้าไปในอีกอาร์เรย์หนึ่ง

อัลกอริทึม 9 (การวิเคราะห์ศัพท์แบบขนาน)
ข้อมูลเข้า DFA $M = (S, \Sigma, \delta, q_0, F)$ และลำดับข้อมูลเข้า $\rho \in \Sigma^*$
ผลลัพธ์: การแบ่งโทเค็น (tokenization) ของ ρ

1. แทนที่สัญลักษณ์ i แต่ละตัวของลำดับข้อมูลเข้า ρ ด้วยรูปแทนแบบอาร์เรย์ของ f_i เรียกลำดับนี้ว่า ϕ
2. ทำ scan บน ϕ โดยใช้การประกอบฟังก์ชันเป็นตัวดำเนินการ scan นี้แทนที่สัญลักษณ์ ρ_i ของลำดับข้อมูลเข้าเดิมด้วยฟังก์ชัน g_i ซึ่งแทนฟังก์ชันการเปลี่ยนสถานะจากสถานะไปยังสถานะสำหรับส่วนนำ (prefix) $\rho_1 \ldots \rho_i$ ของลำดับ สังเกตว่าเราต้องใช้ CREW PRAM ในการทำ scan นี้ (ทำไม?)
3. สร้างลำดับ ψ โดยที่ $\psi_i = g_i(q_0)$ นั่นคือ ใช้สถานะเริ่มต้นของ DFA เป็นดัชนีเข้าไปในอาร์เรย์แต่ละตัวเหล่านี้ ตอนนี้เราได้แทนที่สัญลักษณ์แต่ละตัวด้วยสถานะที่ DFA จะอยู่หลังจากอ่านสัญลักษณ์นั้นแล้ว สถานะที่อยู่ในเซตของสถานะสุดท้ายเป็นตัวกำหนดขอบเขตของโทเค็น

อัลกอริทึมแบบลำดับสำหรับการวิเคราะห์ศัพท์บนลำดับข้อมูลเข้าที่มี n สัญลักษณ์มีความซับซ้อน $T_S(n) = \Theta(n)$ อัลกอริทึมแบบขนานมีความซับซ้อนเชิงขั้น $\Theta(\lg n)$ และความซับซ้อนเชิงงาน $O(n|S|)$ ดังนั้นอัลกอริทึมแบบขนานจึงเร็วกว่า แต่ไม่มีประสิทธิภาพเชิงงาน

Claude อธิบายเพิ่ม: ตัวอย่างเล็ก ๆ และคำตอบของ “ทำไม?”

ให้ DFA มีสองสถานะ $S = \{a, b\}$ แทนฟังก์ชันด้วยอาร์เรย์ $[f(a), f(b)]$ สมมติว่าสัญลักษณ์ `x` ให้ $f_{\mathtt{x}} = [b, b]$ และ `y` ให้ $f_{\mathtt{y}} = [a, b]$ ประกอบ “`x` แล้วตามด้วย `y`” ได้ $[f_{\mathtt{y}}(b), f_{\mathtt{y}}(b)] = [b, b]$ คือแค่ใช้ค่าในอาร์เรย์แรกไปเปิดดูในอาร์เรย์ที่สอง ซึ่งใช้งาน $O(|S|)$ ต่อการประกอบหนึ่งครั้ง งานรวมจึงเป็น $O(n|S|)$
ส่วนเหตุที่ต้องใช้ CREW: ระหว่างประกอบฟังก์ชัน สมาชิกหลายช่องของอาร์เรย์แรกอาจมีค่าสถานะเดียวกัน (ตัวอย่างข้างบน ทั้งสองช่องเป็น b) โพรเซสเซอร์หลายตัวจึงต้อง*อ่าน*ช่องเดียวกันของอาร์เรย์ที่สองพร้อมกัน

5 พลังเชิงเปรียบเทียบของแบบจำลอง PRAM

เรากล่าวว่าแบบจำลอง PRAM P *มีพลังมากกว่า*แบบจำลอง PRAM Q ถ้าอัลกอริทึมที่มีความซับซ้อนเชิงขั้น $S(n)$ บนแบบจำลอง Q มีความซับซ้อนเชิงขั้น $O(S(n))$ บนแบบจำลอง P เราจะแสดงว่าความสามารถในการอ่านและเขียนพร้อมกันช่วยให้แก้ปัญหาบางปัญหาได้เร็วขึ้น จากนั้นจะวัดเชิงปริมาณว่าแบบจำลอง CREW และ CRCW มีพลังมากกว่าแบบจำลอง EREW เพียงใด

5.1 พลังของการอ่านพร้อมกัน

โพรเซสเซอร์ p ตัวของ EREW PRAM ใช้เวลานานเท่าไรในการอ่านค่าหนึ่งค่าในหน่วยความจำร่วม มีวิธีที่เป็นไปได้สองวิธีให้โพรเซสเซอร์ p ตัวอ่านค่าร่วมนั้น วิธีแรก ผลัดกันอ่านค่าแบบลำดับแบบวนรอบ (round robin) ได้ในเวลา $O(p)$ วิธีที่สอง โพรเซสเซอร์ทำสำเนาค่าขณะอ่าน เพื่อให้โพรเซสเซอร์จำนวนมากขึ้นอ่านได้ในรอบถัดไป ถ้าโพรเซสเซอร์แต่ละตัวทำสำเนาหนึ่งชุดขณะอ่าน จำนวนสำเนาจะเพิ่มเป็นสองเท่าทุกรอบ และเวลา $O(\lg p)$ ก็เพียงพอที่จะทำให้ค่านั้นพร้อมให้โพรเซสเซอร์ทั้ง p ตัว

อย่างไรก็ตาม เหตุผลข้างต้นเป็นเพียงการพิสูจน์ขอบเขตบน ต่อไปเราจะให้การพิสูจน์ขอบเขตล่าง คือข้อโต้แย้งว่าจำเป็นต้องใช้ $\Omega(\lg p)$ ขั้น สมมติว่าโพรเซสเซอร์แต่ละตัวทำสำเนาแบบลำดับ k ชุดเมื่ออ่านค่า โดย k อาจเป็นฟังก์ชันของ p แล้วจำนวนสำเนาจะเพิ่มขึ้นเป็น $k(p)$ เท่าในแต่ละรอบ แต่แต่ละรอบใช้เวลา $k(p)$ จำนวนรอบที่ต้องใช้เพื่อทำสำเนา p เท่าคือ $\lg p/\lg k$ และเวลาทั้งหมดคือ $\lg p \cdot \frac{k}{\lg k}$ ซึ่งมากกว่า $\lg p$ เชิงเส้นกำกับ เว้นแต่ $\frac{k}{\lg k} = \Theta(1)$ นั่นคือ k เป็นค่าคงที่ หมายความว่าการพยายามทำสำเนามากขึ้นแบบลำดับในแต่ละรอบไม่ช่วยอะไร สิ่งที่ดีที่สุดที่ทำได้คือทำสำเนาจำนวนคงที่ ซึ่งให้ขอบเขต $\Omega(\lg p)$ ตามที่ต้องการ ตัวคูณการทำสำเนา $k = 2$ ให้ค่าคงที่ที่เล็กที่สุด แต่ค่าคงที่ k ใด ๆ ก็ใช้ได้สำหรับข้อโต้แย้งขอบเขตล่างนี้

5.2 พลังของการเขียนพร้อมกัน

เพื่อแสดงพลังของการเขียนพร้อมกัน เราจะกลับมาพิจารณาปัญหาการหาสมาชิกที่มีค่ามากที่สุดของลำดับ $X = \langle x_1, \ldots, x_n \rangle$ เราเคยเห็นวิธีแก้ปัญหานี้มาแล้วหนึ่งวิธีโดยใช้เทคนิคต้นไม้ทวิภาค ซึ่งได้อัลกอริทึมที่มีประสิทธิภาพเชิงงานสำหรับ EREW PRAM โดยมีความซับซ้อนเชิงงาน $\Theta(n)$ และความซับซ้อนเชิงขั้น $\Theta(\lg n)$ เราจะสร้างอัลกอริทึม CRCW ที่มีความซับซ้อนเชิงขั้นต่ำกว่านี้ได้หรือไม่ คำตอบคือได้ ดังแสดงในอัลกอริทึมต่อไปนี้

อัลกอริทึม 10 (อัลกอริทึม Common-CRCW หรือ Arbitrary-CRCW สำหรับการหาค่ามากที่สุด)
ข้อมูลเข้า: ลำดับ X ที่มีสมาชิก n ตัว
ผลลัพธ์: สมาชิกที่มีค่ามากที่สุดตัวหนึ่งของ X

```
1   integer M[1..n], B[1..n, 1..n]
2   forall i ∈ 1 : n do
3      M[i] ← 1
4   enddo
5   forall i ∈ 1 : n do
6      forall j ∈ 1 : n do
7         B[i, j] ← if X_i ⩾ X_j then 1 else 0 endif
8      enddo
9   enddo
10  forall i ∈ 1 : n do
11     forall j ∈ 1 : n do
12        if not B[i, j] then
13           M[i] ← 0
14        endif
15     enddo
16  enddo
```

ตรวจสอบได้ง่ายว่า $M[i] = 1$ เมื่อจบการคำนวณนี้ก็ต่อเมื่อ X_i เป็นสมาชิกที่มีค่ามากที่สุด การวิเคราะห์อัลกอริทึมนี้แสดงว่า

$S(n) = \Theta(1)$ แต่ $W(n) = \Theta(n^2)$ ดังนั้นอัลกอริทึมนี้เร็วมาก แต่ยังห่างไกลจากการมีประสิทธิภาพเชิงงาน อย่างไรก็ตาม เราอาจต่อเรียงอัลกอริทึมนี้เข้ากับอัลกอริทึมลดรูปหาค่ามากสุดแบบลำดับ หรืออัลกอริทึมลดรูปหาค่ามากสุดบน EREW PRAM เพื่อให้ได้อัลกอริทึม CRCW PRAM ที่มีประสิทธิภาพเชิงงาน โดยมีความซับซ้อนเชิงขั้น $S(n) = \Theta(\lg \lg n)$ ซึ่งเหมาะที่สุดสำหรับแบบจำลอง Common CRCW และ Arbitrary CRCW สังเกตว่ามีอัลกอริทึมที่มีประสิทธิภาพเชิงงานแบบตรงไปตรงมาซึ่งมี $S(n) = \Theta(1)$ สำหรับปัญหาหาค่ามากที่สุดในแบบจำลอง Combining-CRCW ซึ่งแสดงให้เห็นพลังที่เพิ่มขึ้นของแบบจำลองนี้

ความซับซ้อนเชิงขั้น $\Theta(1)$ ของการหาค่ามากสุดในแบบจำลอง CRCW เป็นเรื่องน่าสงสัย และชี้ให้เห็นถึงความไม่สมจริงของแบบจำลอง CRCW PRAM ถึงกระนั้น อัลกอริทึมลดรูปหาค่ามากสุดแบบทีละบิต (bit-serial) ที่อิงแนวคิดคล้ายข้างต้น แต่ใช้เพียงการเขียนพร้อมกันแบบบิตเดียว (เช่น ต้นไม้ “or” แบบเดินสาย: wired “or” tree) ได้พิสูจน์แล้วว่าเร็วมากและใช้งานได้จริงบนเครื่อง SIMD หลายเครื่อง แบบจำลอง CRCW PRAM ถูกใช้สร้างอัลกอริทึมแบบขนานที่ไม่สมจริงโดยสิ้นเชิงได้ง่าย แต่ก็ยังสำคัญ เพราะได้นำไปสู่อัลกอริทึมบางตัวที่ใช้งานได้จริงมากเช่นกัน

Claude อธิบายเพิ่ม: อัลกอริทึม 10 คือ “ทัวร์นาเมนต์พบกันหมด”

ทุกคู่ (i, j) แข่งกันพร้อมกันหมด (n^2 โพรเซสเซอร์) ใครแพ้แม้แต่ครั้งเดียวจะถูกเขียน $M[i] \leftarrow 0$ จุดสำคัญคือหลายโพรเซสเซอร์อาจเขียนลง $M[i]$ เดียวกันพร้อมกัน แต่*ทุกตัวเขียนค่า 0 เหมือนกัน* Common CRCW จึงรับได้ บน EREW การเขียนพร้อมกันนี้ต้องถูกแทนด้วยการลดรูปซึ่งใช้ $\Theta(\lg n)$ ขั้น นี่คือที่มาของช่องว่าง $\Theta(1)$ กับ $\Theta(\lg n)$

5.3 การวัดเชิงปริมาณของพลังการเข้าถึงหน่วยความจำพร้อมกัน

แบบจำลอง CRCW PRAM สมมติว่าการเข้าถึงตำแหน่งเดียวกันในหน่วยความจำพร้อมกันกี่ครั้งก็ได้ สามารถให้บริการได้ในช่วงเวลาเดียว เครื่องขนานจริงย่อมทำแบบนั้นไม่ได้อย่างแน่นอน แบบจำลอง CRCW อาจเป็นแบบจำลองที่ทรงพลังสำหรับการออกแบบอัลกอริทึม แต่ในเชิงสถาปัตยกรรมแล้วเป็นไปได้ยาก เราจะลองดูว่า EREW PRAM ต้องใช้เวลานานเท่าไรในการจำลองการเข้าถึงหน่วยความจำพร้อมกันของ CRCW PRAM โดยเฉพาะ เราจะจัดการกับการเขียนพร้อมกันภายใต้กลยุทธ์ลำดับความสำคัญ (priority) เราขอยอมรับบทตั้งต่อไปนี้โดยไม่พิสูจน์

บทตั้ง 6 (Cole 1988). *EREW PRAM ที่มีโพรเซสเซอร์ p ตัวเรียงลำดับสมาชิก p ตัวได้ใน $\Theta(\lg p)$ ขั้น* □

จากบทตั้งนี้ เราพิสูจน์ทฤษฎีบทต่อไปนี้ได้

ทฤษฎีบท 4. *EREW PRAM ที่มีโพรเซสเซอร์ p ตัวจำลอง Priority CRCW PRAM ที่มีโพรเซสเซอร์ p ตัวได้ โดยช้าลง (slowdown) $\Theta(\lg p)$ เท่า*

พิสูจน์. อันที่จริง สิ่งที่เราจะแสดงทั้งหมดคือการจำลองที่รับประกันการช้าลง $O(\lg p)$ ขอบเขตล่าง $\Omega(\lg p)$ เป็นจริง แต่การพิสูจน์อยู่นอกขอบเขตของวิชานี้ ดูบทที่ 10 ของ JáJá หากสนใจ

สมมติว่า CRCW PRAM มีโพรเซสเซอร์ p ตัว คือ P_1 ถึง P_p และมีตำแหน่งหน่วยความจำ m ตำแหน่ง คือ M_1 ถึง M_m และ EREW PRAM มีจำนวนโพรเซสเซอร์เท่ากันแต่มีตำแหน่งหน่วยความจำเพิ่มอีก $O(p)$ ตำแหน่ง เราจะแสดงวิธีจำลองขั้น CR หรือ CW ซึ่งโพรเซสเซอร์ P_i เข้าถึงตำแหน่งหน่วยความจำ M_j บน EREW PRAM ในการจำลองของเรา เราจะใช้อาร์เรย์ T เก็บคู่ (j, i) และใช้อาร์เรย์ S บันทึกว่าโพรเซสเซอร์ตัวใดได้สิทธิ์เข้าถึงตำแหน่งหน่วยความจำในท้ายที่สุด

โค้ดต่อไปนี้อธิบายการจำลอง

อัลกอริทึม 11 (การจำลอง Priority CRCW PRAM ด้วย EREW PRAM)
ข้อมูลเข้าชั้นการเข้าถึงหน่วยความจำของ CRCW ซึ่งโพรเซสเซอร์ P_i อ่าน/เขียนตำแหน่งหน่วยความจำร่วม M_j โดย $i \in 1:p$ และ $j \in 1:m$ สำหรับขั้นการอ่าน P_i ได้รับค่าที่เก็บอยู่ที่ M_j สำหรับขั้นการเขียน ค่าที่ถูกเขียนลง M_j คือค่าที่โพรเซสเซอร์ P_k เขียน เมื่อ $k = \min\{i \mid P_i \text{ เขียน } M_j\}$
ผลลัพธ์: การจำลองขั้น CRCW บน EREW PRAM ที่มีโพรเซสเซอร์ p ตัวและหน่วยความจำร่วม $m + p$ ตำแหน่ง

1 forall $i \in 1:p$ do
2 โพรเซสเซอร์ P_i เขียนคู่ (j, i) ลงใน $T[i]$
3 enddo
4 เรียง T ตาม j ก่อน แล้วจึงตาม i
5 forall $k \in 1:p$ do
6 โพรเซสเซอร์ P_1 อ่าน $T[1] = (j_1, i_1)$ และตั้ง $S[i_1]$ เป็น 1 สำหรับ $k > 1$ โพรเซสเซอร์ P_k อ่าน $T[k] = (j_k, i_k)$ และ $T[k-1] = (j_{k-1}, i_{k-1})$[แก้ 4] แล้วตั้ง $S[i_k]$ เป็น 1 ถ้า $j_k \neq j_{k-1}$ และเป็น 0 ในกรณีอื่น
7 enddo
8 forall $i \in 1:p$ do
9 สำหรับ CW โพรเซสเซอร์ P_i เขียนค่าของตนลง M_j ถ้า $S[i] = 1$
10 สำหรับ CR โพรเซสเซอร์ P_i อ่านค่าใน M_j ถ้า $S[i] = 1$ แล้วทำสำเนาค่านั้นให้โพรเซสเซอร์อื่นในเวลา $O(\lg p)$
11 enddo

บรรทัดที่ 1 ใช้ $\Theta(1)$ ขั้น จากบทตั้งที่ 6 บรรทัดที่ 4 ใช้ $\Theta(\lg p)$ ขั้น บรรทัดที่ 5 ใช้ $\Theta(1)$ ขั้นอีกเช่นกัน และบรรทัดที่ 8 ใช้ $\Theta(1)$ ขั้นสำหรับการเขียนพร้อมกัน และ $O(\lg p)$ ขั้นสำหรับการอ่านพร้อมกัน ดังนั้นการจำลองขั้น CRCW จึงทำงานใน $O(\lg p)$ ขั้นของ EREW

รูปที่ 7 แสดงตัวอย่างของการจำลองนี้ □

โพรเซสเซอร์ =	1	2	3	4	5
ตำแหน่งหน่วยความจำที่เข้าถึง =	3	3	7	3	7
ค่าที่เขียน =	6	2	9	3	7
T =	$(3,1)$	$(3,2)$	$(7,3)$	$(3,4)$	$(7,5)$
T ที่เรียงแล้ว =	$(3,1)$	$(3,2)$	$(3,4)$	$(7,3)$	$(7,5)$
S =	1	0	1	0	0

รูปที่ 7: ภาพประกอบการจำลองการเขียนพร้อมกันด้วย EREW PRAM ที่มีโพรเซสเซอร์ห้าตัว โพรเซสเซอร์ P_1 เขียนค่า 6 ลงตำแหน่งหน่วยความจำ M_3 ได้สำเร็จ และโพรเซสเซอร์ P_3 เขียนค่า 9 ลงตำแหน่งหน่วยความจำ M_7 ได้สำเร็จ

Claude อธิบายเพิ่ม: ข้อสังเกตเกี่ยวกับตาราง S ในรูปที่ 7

แถว S ในรูปเรียงตามหมายเลขโพรเซสเซอร์ i (ไม่ใช่ตามลำดับของ T ที่เรียงแล้ว) ตามที่บรรทัดที่ 6 ตั้งค่า $S[i_k]$ หลังเรียง T ตัวแรกของกลุ่ม $j = 3$ คือ $(3,1)$ และตัวแรกของกลุ่ม $j = 7$ คือ $(7,3)$ จึงได้ $S[1] = S[3] = 1$ ซึ่งตรงกับกติกา Priority: ในบรรดาผู้เขียนตำแหน่งเดียวกัน โพรเซสเซอร์ที่มีหมายเลขน้อยที่สุดชนะ เพราะการเรียงตาม i เป็นลำดับรองทำให้ผู้ชนะอยู่หัวกลุ่มเสมอ

5.4 ความสัมพันธ์ระหว่างแบบจำลอง PRAM กับการคำนวณแบบขนานในทางปฏิบัติ

แบบจำลอง PRAM มักถูกวิจารณ์ว่าไม่สามารถสร้างขึ้นจริงได้อย่างสมเหตุสมผลเมื่อใช้โพรเซสเซอร์มากกว่าไม่กี่ตัว ดังนั้นอัลกอริทึมบน PRAM จึงขยายขนาดในทางปฏิบัติไม่ได้อย่างที่มาตรวัดความซับซ้อนบ่งบอก ปัญหาหลักคือสมมติฐานของ PRAM ที่ว่าการอ้างอิงหน่วยความจำแบบขนานมีเวลาแฝง (latency) คงที่ สมมติฐานนี้จะไม่สมจริงอย่างรวดเร็วเมื่อเราเพิ่มจำนวนโพรเซสเซอร์ p ด้วยเหตุผลต่อไปนี้

1. ส่วนประกอบหน่วยความจำในปัจจุบันให้บริการสายการอ้างอิงแบบลำดับได้ด้วยอัตราสูงสุดค่าหนึ่ง เมื่อ p เพิ่มขึ้น ระบบหน่วยความจำต้องส่งข้อมูลต่อหน่วยเวลาเพิ่มขึ้น ซึ่งหมายความว่าในที่สุดต้องใช้ส่วนประกอบหน่วยความจำ (แบงก์: banks) $\Omega(p)$ ชุด การเชื่อมต่อโพรเซสเซอร์และแบงก์หน่วยความจำเหล่านี้พร้อมกันต้องใช้เครือข่ายสวิตชิง (switching network) ซึ่งไม่อาจมีเวลาแฝง $\Theta(1)$ ได้

2. ส่วนประกอบหน่วยความจำในปัจจุบันอนุญาตให้อ่านพร้อมกันได้ไม่เกินจำนวนคงที่ และดังที่เราเห็นในสองหัวข้อก่อนหน้า นี่หมายความว่ามีเวลาแฝง $\Omega(\lg p)$ ในการให้บริการการอ่านและเขียนพร้อมกันโดยใช้ส่วนประกอบหน่วยความจำเหล่านี้

3. เมื่อ p เพิ่มขึ้น ปริมาตรทางกายภาพของเครื่องจริงต้องเพิ่มขึ้น ดังนั้นตำแหน่งหน่วยความจำอย่างน้อยบางตำแหน่งต้องอยู่ห่างจากโพรเซสเซอร์บางตัวมากขึ้น ดังนั้นเวลาแฝงของหน่วยความจำ PRAM ต้องเพิ่มขึ้นตาม $\Omega(\sqrt[3]{p})$ อันที่จริง ข้อโต้แย้งคล้ายกันแสดงว่าเราไม่สามารถสร้างการอ้างอิงหน่วยความจำที่ใช้เวลาคงที่ได้แม้แต่ในแบบจำลอง RAM อย่างง่าย เมื่อเราเพิ่มขนาดหน่วยความจำ นี่คือเหตุผลที่แคชมีอยู่ทั่วไป และเหตุผลที่แบบจำลอง RAM ไม่ใช่แบบจำลองต้นทุนที่แม่นยำสมบูรณ์สำหรับการคำนวณแบบลำดับ แต่สำหรับ PRAM การมี “ของ” ที่ต้องขยายขนาดมากขึ้นมาก (โพรเซสเซอร์ เครือข่ายเชื่อมต่อ แบงก์หน่วยความจำ) ทำให้เวลาแฝงเพิ่มขึ้นตาม p อย่างรุนแรงยิ่งขึ้น

ถึงกระนั้น อัลกอริทึมบน PRAM ก็อาจเป็นจุดเริ่มต้นที่มีคุณค่าสำหรับการสร้างระบบขนานที่ใช้งานได้จริง ตัวอย่างเช่น อัลกอริทึมใด ๆ ที่ทำงานได้อย่างมีประสิทธิภาพบนแบบจำลอง PRAM ที่มีโพรเซสเซอร์ p ตัว สามารถแปลงเป็นอัลกอริทึมที่ทำงานได้อย่างมีประสิทธิภาพบนเครื่องที่มีโพรเซสเซอร์ p/L ตัว ซึ่งมีระบบหน่วยความจำที่มีเวลาแฝง L ได้ ซึ่งเป็นเครื่องที่สมจริงกว่า PRAM มาก ในอัลกอริทึมที่แปลงแล้ว โพรเซสเซอร์แต่ละตัวจาก p/L ตัวจะจำลองโพรเซสเซอร์ PRAM L ตัว เวลาแฝงของหน่วยความจำถูก “ซ่อน” ไว้ เพราะโพรเซสเซอร์มีงานที่มีประโยชน์และเป็นอิสระต่อกัน L หน่วยให้ทำขณะรอให้การเข้าถึงหน่วยความจำเสร็จสิ้น โพรเซสเซอร์เวกเตอร์แบบขนาน (parallel vector processors) หรือ Tera MTA (สถาปัตยกรรมแบบหลายเธรด: multi-threaded architecture) ที่เพิ่งออกมาเป็นตัวอย่างของเครื่องขนานแบบหน่วยความจำร่วมที่เหมาะกับการแปลงแบบนี้

ในกรณีอื่น อัลกอริทึมบน PRAM อาจจัดเรียงใหม่ให้ลดความต้องการด้านการประสานจังหวะและการเข้าถึงหน่วยความจำร่วม เพื่อให้เข้ากันได้ดีขึ้นกับมัลติโพรเซสเซอร์แบบหน่วยความจำร่วมที่ใช้โพรเซสเซอร์ทั่วไปที่มีแคช นี่คือหัวข้อของหน่วยถัดไปในวิชานี้

Claude อธิบายเพิ่ม: สรุปภาพรวม

- **มาตรวัดสองตัว:** งาน $W(n)$ (ต้องไม่เกินอัลกอริทึมแบบลำดับที่ดีที่สุด) และจำนวนขั้น $S(n)$ (ยิ่งน้อยยิ่งใช้โพรเซสเซอร์ได้มาก) ทฤษฎีบทของเบรนต์เชื่อมทั้งสองเข้ากับเครื่องจริงด้วย $W/p + S$
- **กล่องเครื่องมือ:** ต้นไม้สมดุล (scan) · กระโดดตัวชี้ (ลิสต์/ต้นไม้) · ต่อเรียงอัลกอริทึม (ซ่อมงานที่เกิน) · ทัวร์ออยเลอร์ (ต้นไม้ → ลิสต์) · แบ่งแยกและเอาชนะ · สุ่มเพื่อทำลายความสมมาตร
- **ราคาของความสมจริง:** CRCW → EREW ช้าลงได้ถึง $\Theta(\lg p)$ เท่า และเครื่องจริงยังมีเวลาแฝงของหน่วยความจำที่โตตาม p วิธีรับมือคือมี*ความขนานเกินกว่าจำนวนโพรเซสเซอร์* (ใช้ p/L โพรเซสเซอร์จำลอง p ตัว) เพื่อซ่อนเวลาแฝง

ภาคผนวก: รายการแก้คำผิดจากต้นฉบับ

ภาคผนวกนี้ไม่ใช่ส่วนของต้นฉบับ แต่ Claude รวบรวมไว้เอง จุดที่แก้ทั้งหมดมีเครื่องหมาย [แก้ n] กำกับในเนื้อหา หมายเลขหน้าที่อ้างถึงเป็นหน้าของต้นฉบับ (`pram.pdf`)

[แก้ 1] บทพิสูจน์ทฤษฎีบทที่ 1 (ทฤษฎีบทของเบรนต์) หน้า 5 ของต้นฉบับ

ต้นฉบับ: "the time to simulate step $W_i(n)$ of the WT algorithm"

แก้เป็น: "the time to simulate step i" (เวลาที่ใช้จำลอง*ขั้นที่* i)

เหตุผล: $W_i(n)$ คือ*จำนวนการดำเนินการ*ในขั้นที่ i ไม่ใช่ชื่อของขั้น ต้นฉบับเอาปริมาณงานมาใช้แทนหมายเลขขั้น ประโยคเดียวกันก่อนหน้านั้นเองก็นิยามไว้ว่า "For each time step i ... let $W_i(n)$ be the number of operations in that step"

[แก้ 2] อัลกอริทึม 8 (random mate) บรรทัดที่ 6 หน้า 16 ของต้นฉบับ

ต้นฉบับ: $\text{gender}[v] = \text{rand}(\{\text{M}, \text{F}\})$

แก้เป็น: $\text{gender}[v] \leftarrow \text{rand}(\{\text{M}, \text{F}\})$

เหตุผล: บรรทัดนี้เป็น*การกำหนดค่า* (สุ่มเพศแล้วเก็บไว้) แต่ต้นฉบับใช้ $=$ ซึ่งในสัญกรณ์ของเอกสารนี้หมายถึง*การเปรียบเทียบ* (เช่น if $\text{gender}[\ldots] = \text{M}$ ในบรรทัดที่ 9 และ 12) ส่วนการกำหนดค่าบรรทัดอื่นทั้งเล่มใช้ $\leftarrow$

[แก้ 3] บทพิสูจน์ทฤษฎีบทที่ 3 หน้า 17 ของต้นฉบับ

ต้นฉบับ: "the probability of still having to work after $5 \lg |V|$ steps"

แก้เป็น: "after $5 \lg |V|$ iterations" (หลังการวน $5 \lg |V|$ *รอบ*)

เหตุผล: p_k และตัวทฤษฎีบทนิยามทุกอย่างเป็น*จำนวนรอบ*ของลูป while ("after $5 \lg |V|$ iterations") ไม่ใช่จำนวนขั้น แต่ละรอบใช้ $\Theta(1)$ ขั้นตามบทตั้งที่ 2 จึงไม่ใช่ปริมาณเดียวกัน แม้จะต่างกันแค่ค่าคงที่ก็ตาม

[แก้ 4] อัลกอริทึม 11 (การจำลอง Priority CRCW) บรรทัดที่ 6 หน้า 21 ของต้นฉบับ

ต้นฉบับ: $T_{k-1} = [j_{k-1}, i_{k-1}]$

แก้เป็น: $T[k-1] = (j_{k-1}, i_{k-1})$

เหตุผล: ผิดสองจุด: (1) T เป็นอาร์เรย์ ทั้งบรรทัดเดียวกันและบรรทัดที่ 2 อ้างถึงสมาชิกแบบ $T[1]$, $T[k]$, $T[i]$ แต่ตรงนี้เขียนเป็นตัวห้อย T_{k-1} (2) สมาชิกของ T เป็น*คู่อันดับ* (j, i) ตามบรรทัดที่ 2 ("writes the pair (j, i)") และรูปที่ 7 แต่ตรงนี้ใช้วงเล็บเหลี่ยม $[\cdot]$ ซึ่งในเอกสารนี้ใช้สำหรับการอ้างดัชนีอาร์เรย์

ข้อสังเกตเพิ่มเติม (ไม่ได้แก้ในเนื้อหา)

- **อัลกอริทึม 11 บรรทัดที่ 6 บน EREW:** เมื่อ P_k อ่าน $T[k-1]$ โพรเซสเซอร์ P_{k-1} ก็อ่าน $T[k-1]$ ในขั้นเดียวกัน จึงเป็นการอ่านพร้อมกันซึ่ง EREW ไม่อนุญาต อันนี้ไม่ใช่คำผิด และไม่กระทบผลลัพธ์: แค่แยกเป็นสองขั้นย่อย (ทุกตัวอ่าน $T[k]$ ก่อน แล้วจึงอ่าน $T[k-1]$) ก็ยังใช้ $\Theta(1)$ ขั้นเท่าเดิม
- **คำผิดภาษาอังกฤษที่หายไปเองในการแปล** (ไม่กระทบความหมาย): "must insure" $\rightarrow$ ensure (หน้า 2), "are are in this memory" (หน้า 3), "$S(n)$ and $W(n)$ for of Algorithm 2" (หน้า 4), "it is lives" (หน้า 17), "simutaneously" (หน้า 22)